

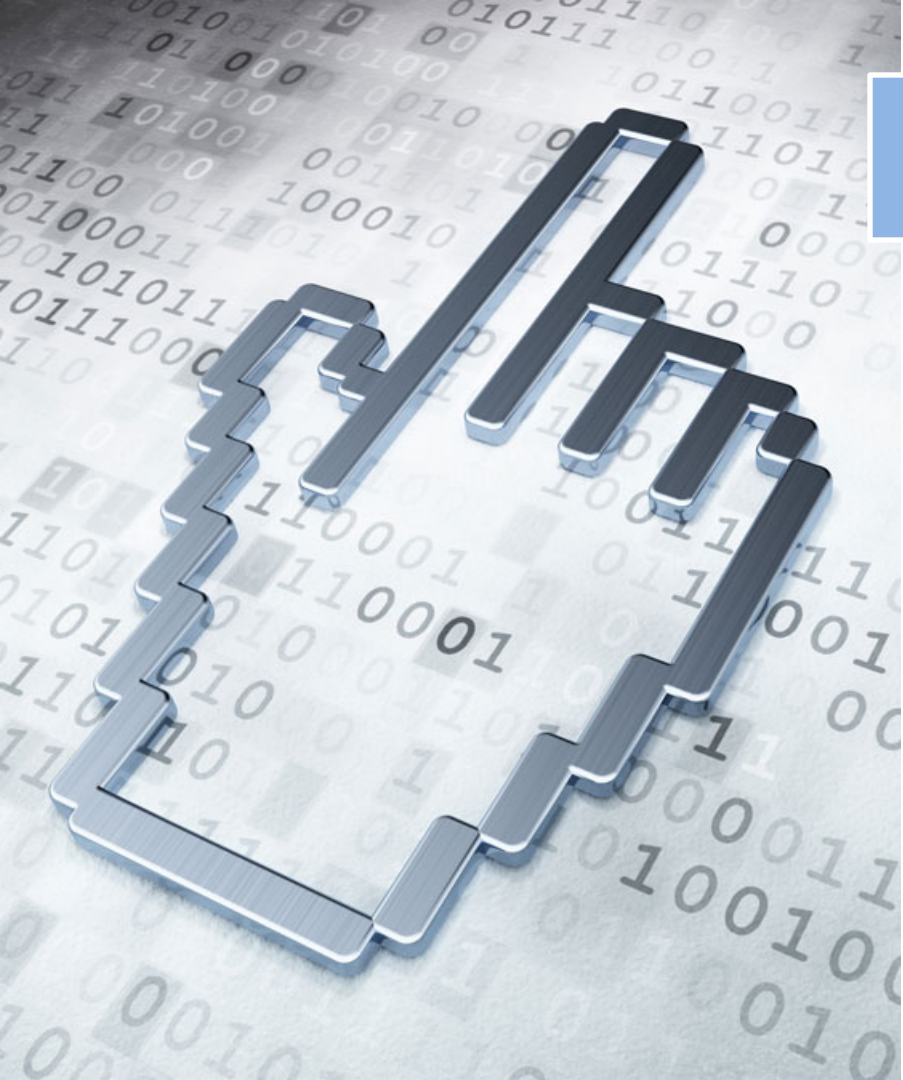


编译原理

第一章 绪论



哈尔滨工业大学 陈鄞 单丽莉



本章内容

1.1 什么是编译

1.2 编译系统的结构

1.3 编译器的生成

1.4 为什么要学习编译原理

1.5 编译技术的应用

1.1 什么是编译?

类似于数学定义或
自然语言的简洁形式

- 接近人类表达习惯
- 不依赖于特定机器
- 编写效率高

高级语言

(High Level Language)

引入助记符

- 依赖于特定机器，
非计算机专业人员
使用受限制

汇编语言

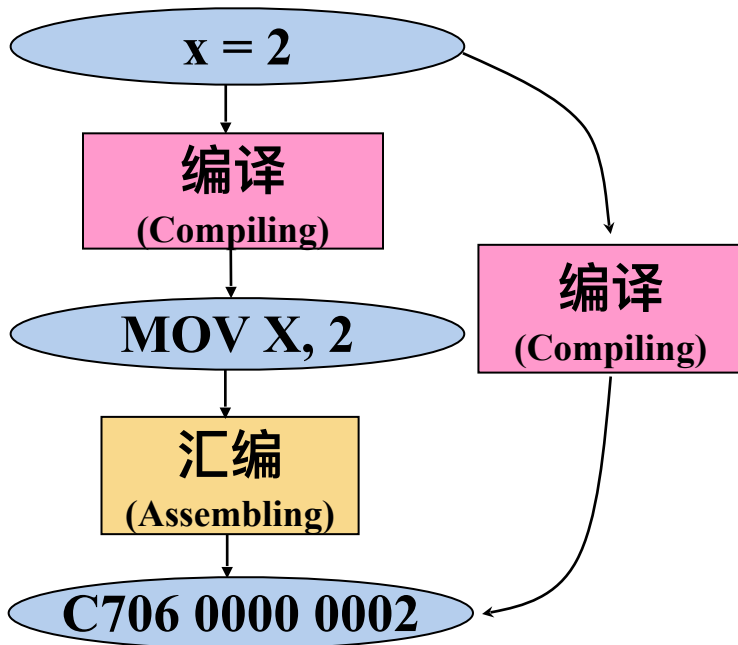
(Assembly Language)

编写效率依然很低
可以被计算机直接执行

- 与人类表达习惯
相去甚远
- 难记忆
- 难编写、难阅读
- 易写错

机器语言

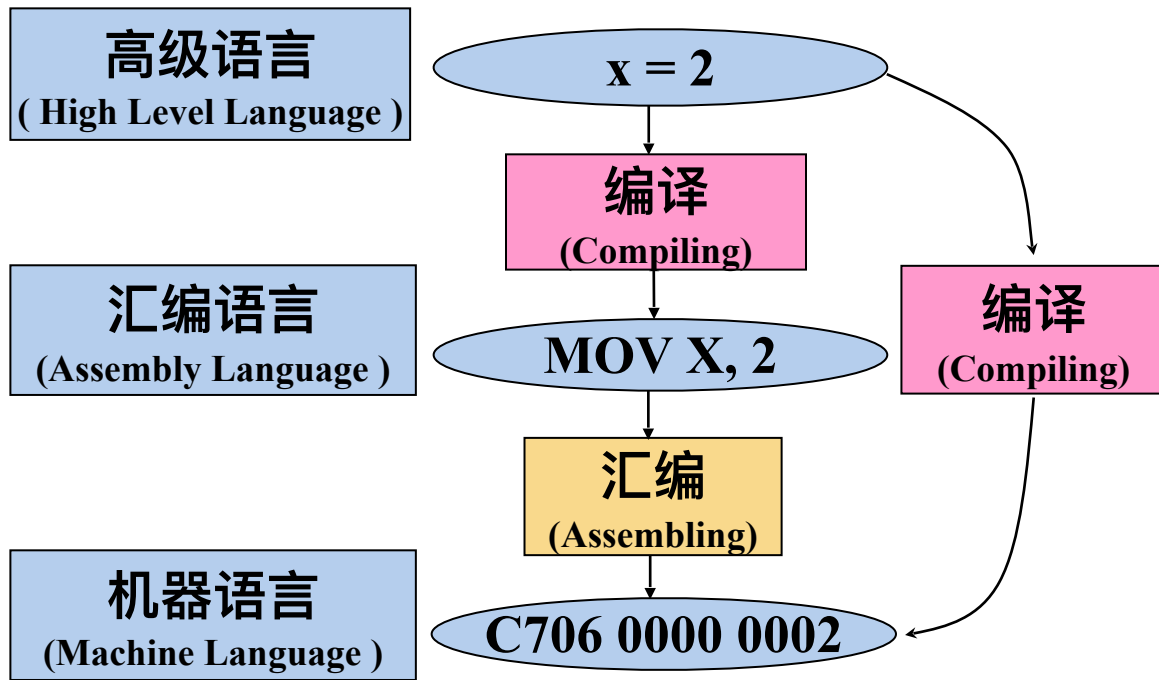
(Machine Language)



不同硬件平台，
不同的操作系统，编译器也不同。

编译的主要任务：翻译 和 优化

优化目标：代码规模小、执行速度快

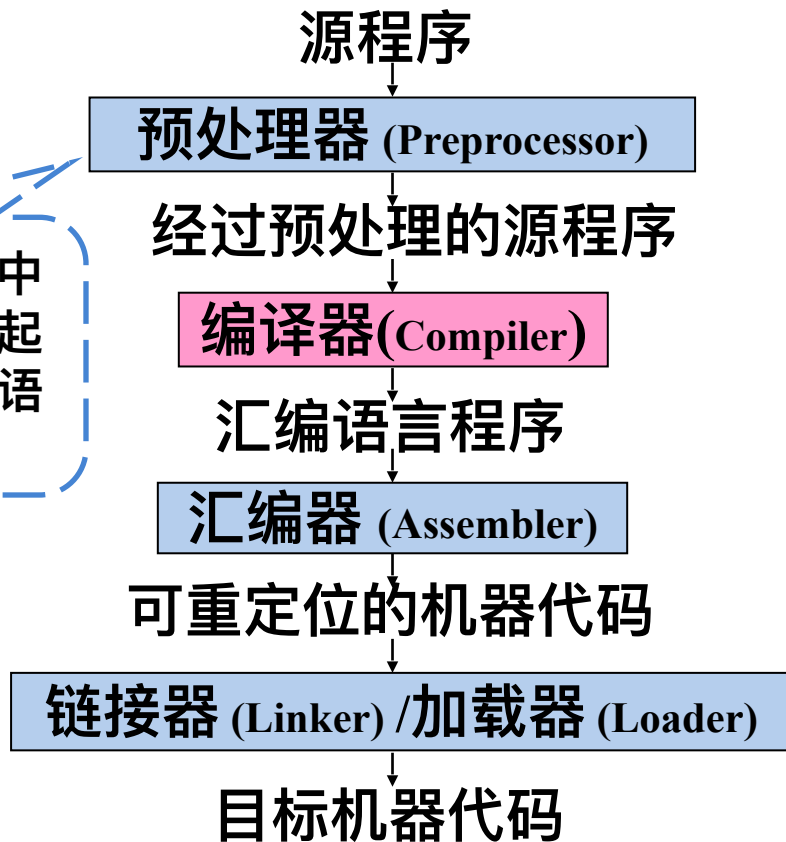


编译：将高级语言翻译成汇编语言或机器语言的过程

源语言 目标语言

编译器在语言处理系统中的位置

- 把存储在不同文件中的源程序**聚合**在一起
- 把被称为**宏**的缩写语句转换为原始语句



编译器在语言处理系统中的位置

源程序

预处理器 (Preprocessor)

经过预处理的源程序

编译器 (Compiler)

汇编语言程序

汇编器 (Assembler)

可重定位的机器代码

链接器 (Linker) / 加载器 (Loader)

起始位置 + 相对地址 = 绝对地址 目标机器代码

可重定位

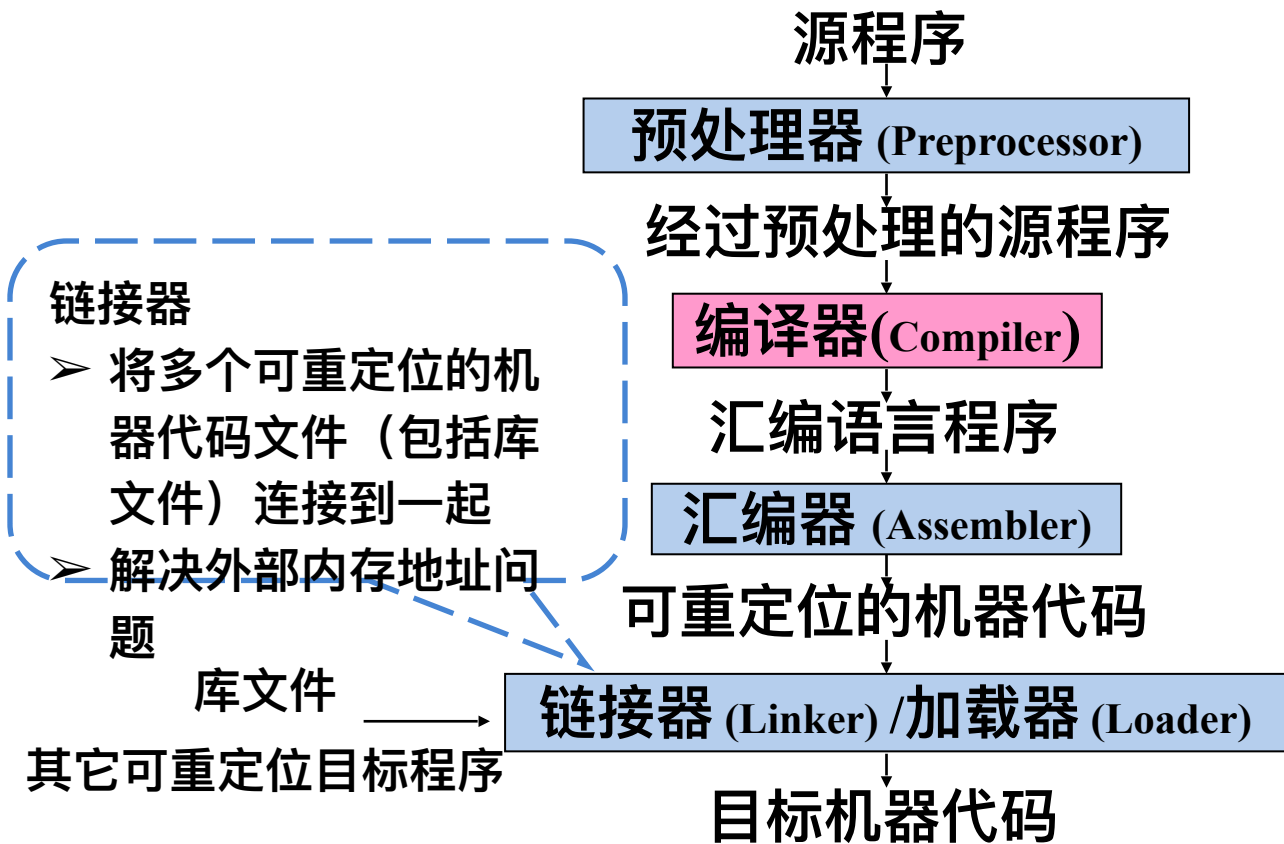
(Relocatable):

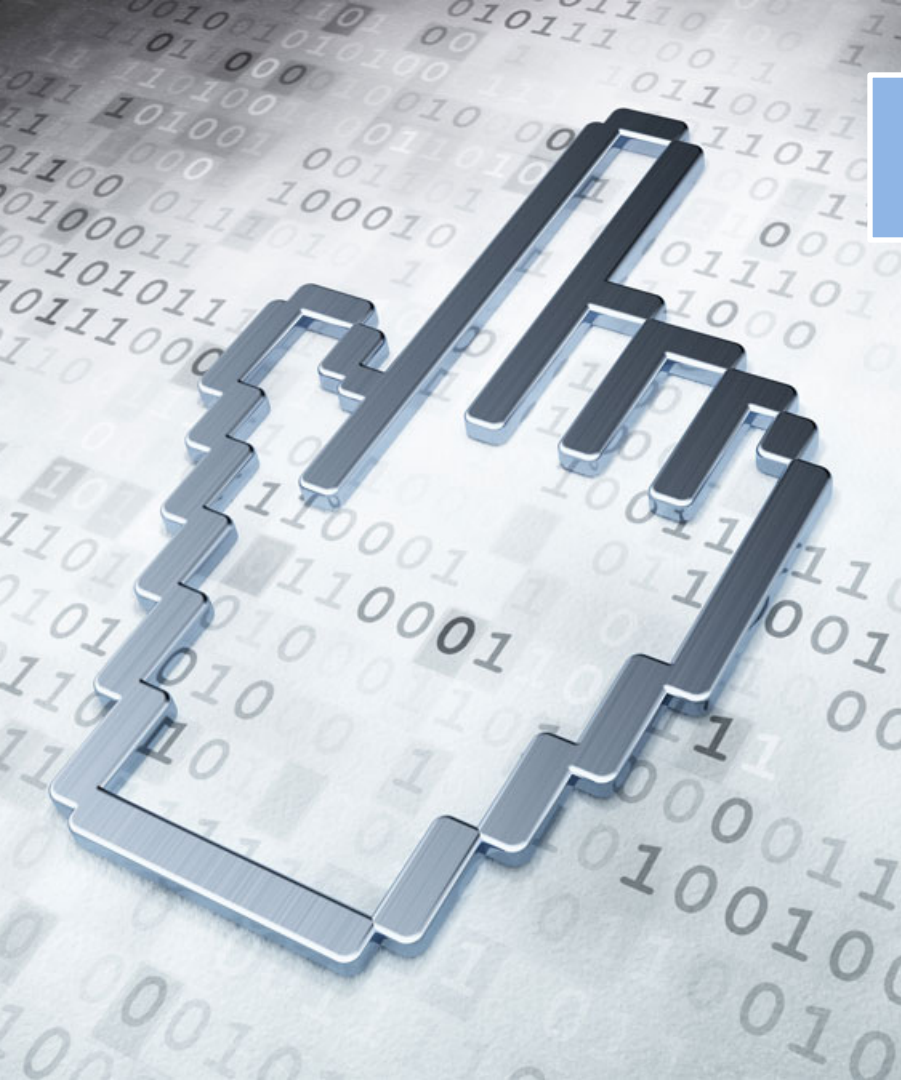
在内存中存放的起始
位置 L 不是固定的

加载器:

修改可重定位地址;
将修改后的指令和数据
放到内存中适当的位置

编译器在语言处理系统中的位置





本章内容

1.1 什么是编译

1.2 编译系统的结构

1.3 编译器的生成

1.4 为什么要学习编译原理

1.5 编译技术的应用

1.2 编译系统的结构

```
#include<stdio.h>
int main()
{
    int a,b,ge,shi,bai,m,n,i,number;
    printf("请输入(输入完两个数按一次回车键): \n");
    scanf("%d %d",&a,&b);
    while(a!=0,b!=0)
        scanf("%d %d",&a,&b);
    if(a>=100,b<=999)
        if(a>b)
            m=b,n=a;
        else
            m=a,n=b;
    for(i=m;i<=n;i++)
    {
        bai=i/100;
        shi=(i%100)/10;
        ge=i%10;
        if(i==bai*bai*bai+shi*shi*shi+ge*ge*ge)
            printf("%5d",i);
        number++;
        printf("\n");
    }
    if(number==0)
        printf("no\n");
}
```

机器是如何自动翻译
的？

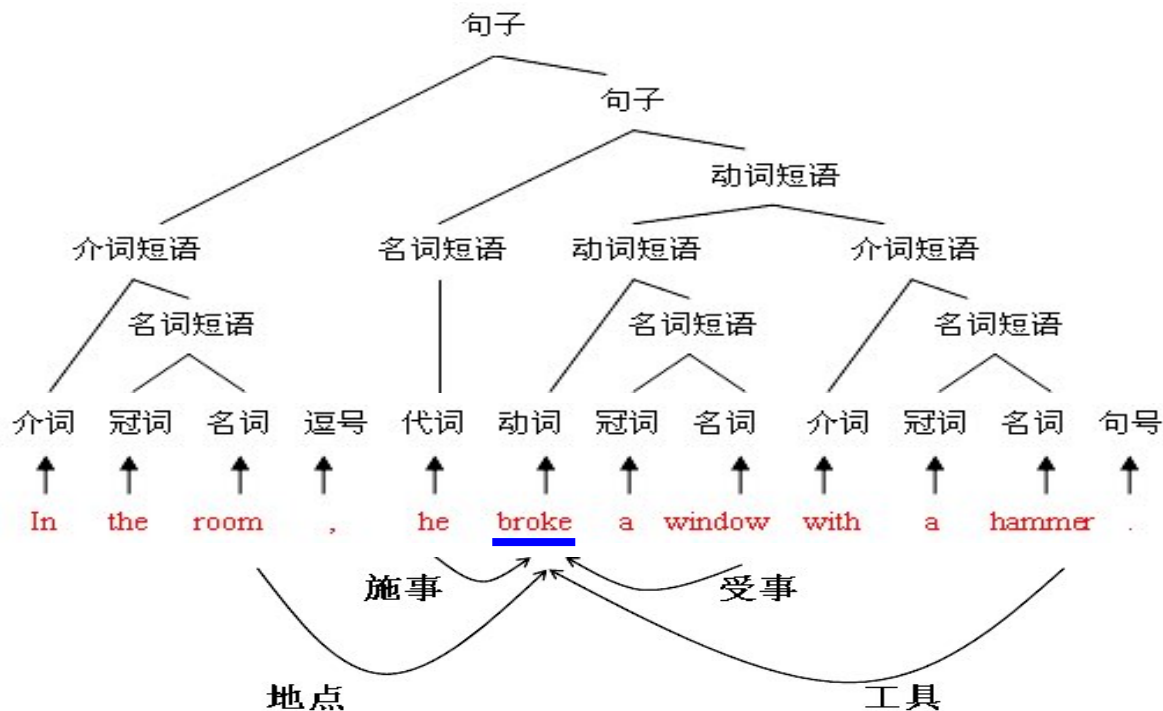
编译器

高级语言程序

汇编语言程序/机器语言程序

```
59    call sc
60    mov  al, [si - 1]
61    call sc
62    mov  al, [si]
63    call sc
64    mov  al, ' '
65    call sc
66    ret
67 storechr endp
68 ;清屏, 无入口参数
69 clearcrt proc near
70     mov  ax, 0600h
71     mov  bh, 07h
72     mov  cx, 00h
73     mov  dx, 184fh
74     int  10h
75
84     mov  ah, 02h
85     mov  dl, cr
86     int  21h
87     mov  ah, 02h
88     mov  dl, lf
89     int  21h
90     .untilcxz
91     ret
92 nextlien endp
93     end
94
```


In the room , he broke a window with a hammer.



初步翻译
在房间里，他砸
了一扇窗用锤
子。

润色

他在房间里用锤子砸了一扇窗。

1.2 编译系统的结构

In the room , he broke
a window with a
hammer.



他在房间里用锤子砸了一
扇窗。

把英文翻译为中文

编译程序工作阶段

1. 识别出文本中的一个单词
2. 分析句子的语法结构
3. 根据句子的含义进行初步翻译
4. 对译文进行润色
5. 写出最后的译文

词法分析



语法分析



语义分析与中间代码生成



优化

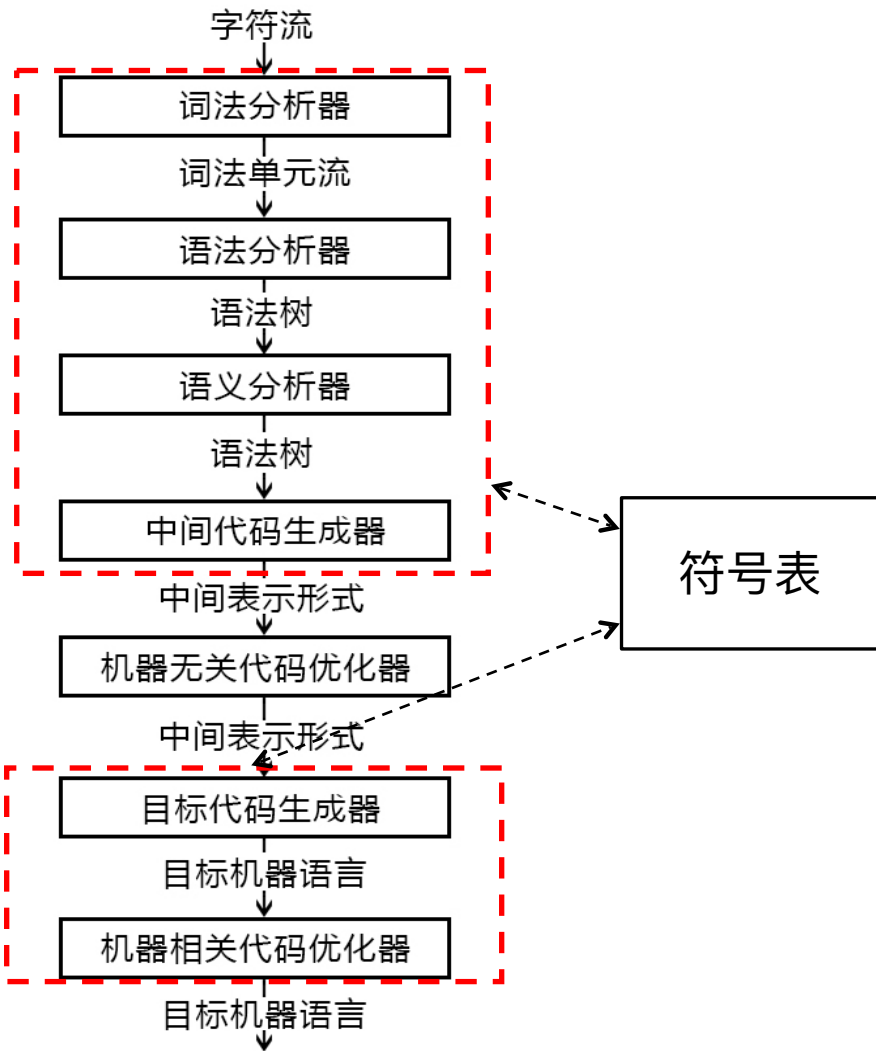


目标代码产生

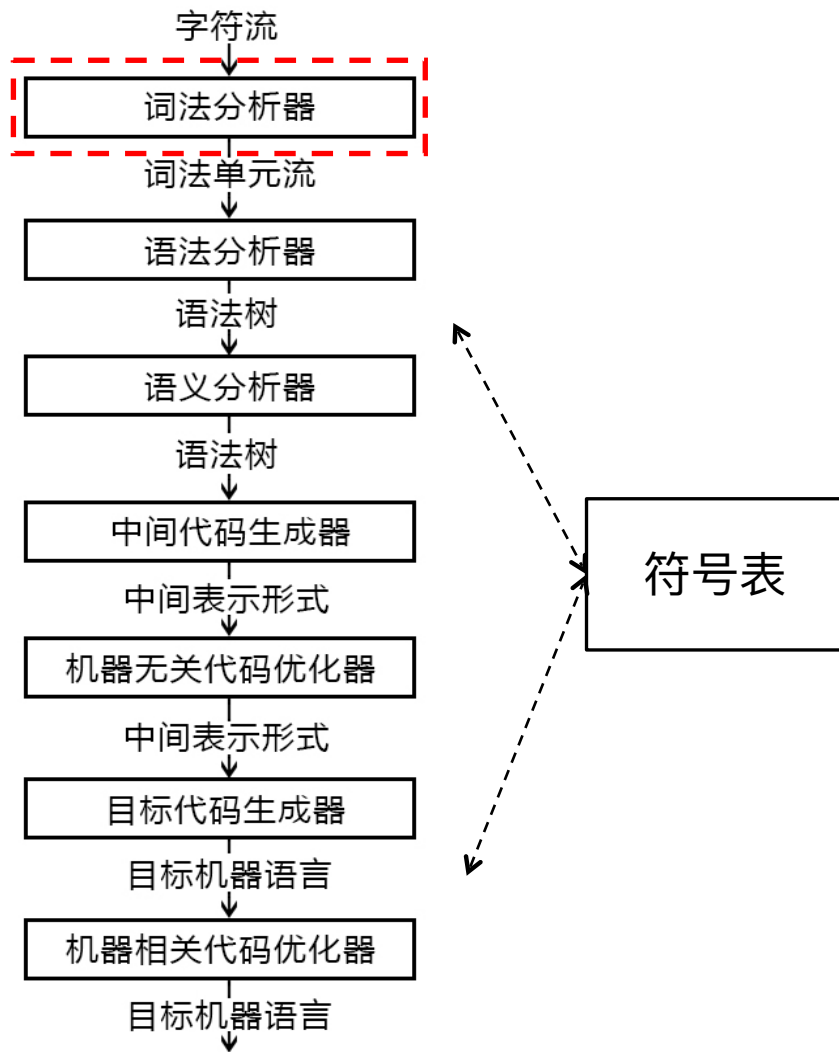
编译器的结构

分析部分/
前端(front
end):
与源语言相关
与目标语言无关

综合部分/
后端(back
end):
与目标语言相关



编译器的结构



词法分析/扫描(*Scanning*)

- 词法分析由词法分析器(**Lexical Analyzer**)完成，又称为扫描器 **Scanner** 。
- 任务：
 - 从左到右扫描源程序字符串，识别出一个个单词，并转换成符号表示的单词(token)串；同时要检查词法错误。
- 输入：源程序字符串(字符流)
- 输出：单词串 (token序列，词法单元序列，符号串)
 - 每个单词(词法单元)：
二元组序对：<种别码，属性值>

例：词法分析后得到的符号表示的单词串

➤ 输入 **while(value!=100){ num++; }**

➤ 输出

1	while
2	(
3	value
4	!=
5	100
6	)
7	{
8	num
9	++
10	;
11	}

词素

< WHILE ,	-	>
< (,	-	>
< ID , value	-	>
< != ,	-	>
< CONST , 100	-	>
<) ,	-	>
< { ,	-	>
< ID , num	-	>
< ++ ,	-	>
< ; ,	-	>
< } ,	-	>

输出符号串

二元组：
<种别码, 属性值>

➤ 细分任务

★ 编码设计：代码中的**单词**与 **种别码(token)** 的 **对应关系**。单词必须编码为**语法分析器**能处理的符号。

★ 单词表示和识别：使用正则文法来形式化描述，设计DFA来识别**单词**。
词法分析依赖单词构成的规则，正则文法或正则表达式。

token: < 种别码, 属性值 >

1	while	< WHILE , - >
2	(	< (, - >
3	value	< ID , value >
4	!=	< != , - >
5	100	< CONST , 100 >
6	)	<) , - >
7	{	< { , - >
8	num	< ID , num >
9	++	< ++ , - >
10	;	< ; , - >
11	}	< } , - >

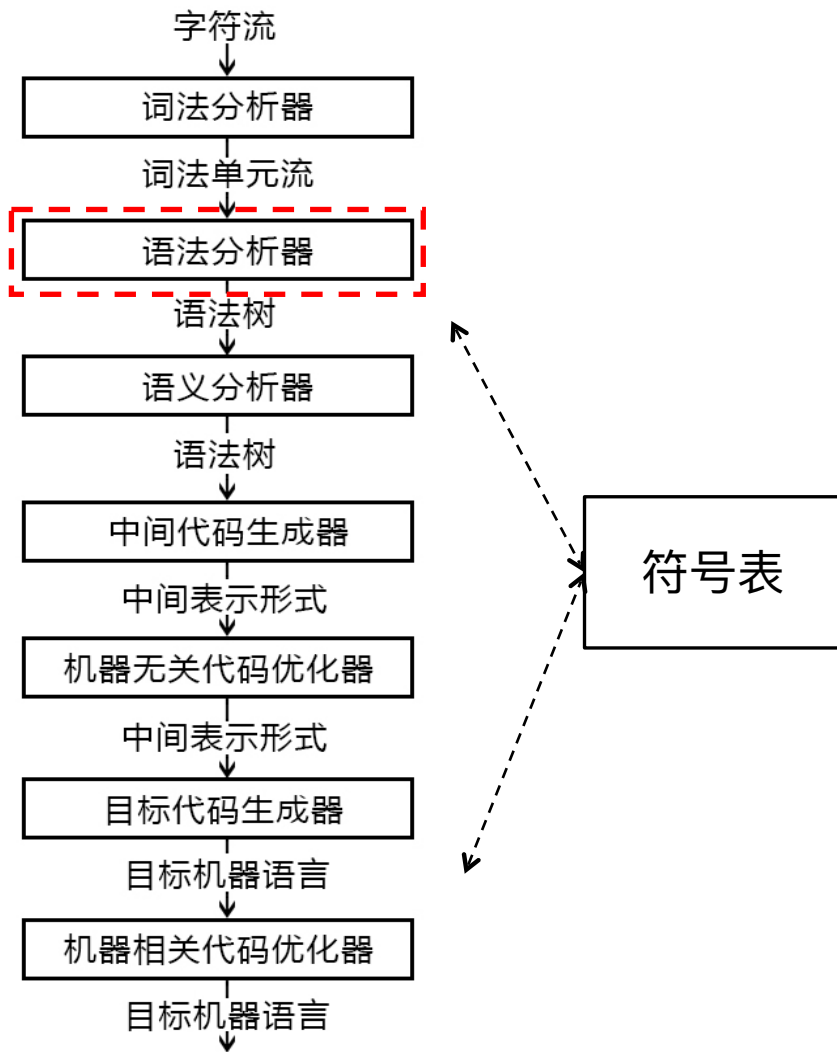
编译器的结构

➤ 语法分析器：

➤ 功能：组词成句

➤ 输入：词法单元序列

➤ 输出：语法分析树



- ## 任务：组词成句

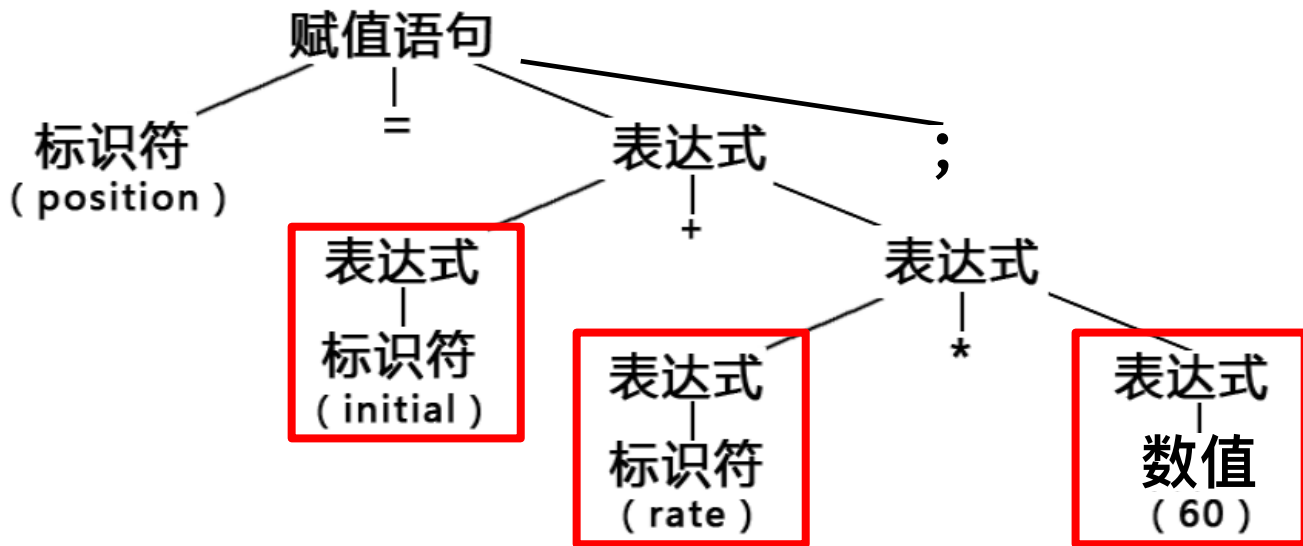


输出：语法成分 (语法分析树)

例1：赋值语句的分析树

position = initial + rate * 60 ;

$\langle \text{标识符, position} \rangle \Rightarrow \langle \text{标识符, initial} \rangle \langle + \rangle \langle \text{标识符, rate} \rangle \langle * \rangle \langle \text{数值, 60} \rangle \langle ; \rangle$



例2：变量声明语句的分析树

➤文法：

$D \rightarrow T IDS;$

$T \rightarrow \text{int} \mid \text{real} \mid \text{char} \mid \text{bool}$

$IDS \rightarrow \text{id} \mid IDS, \text{id}$

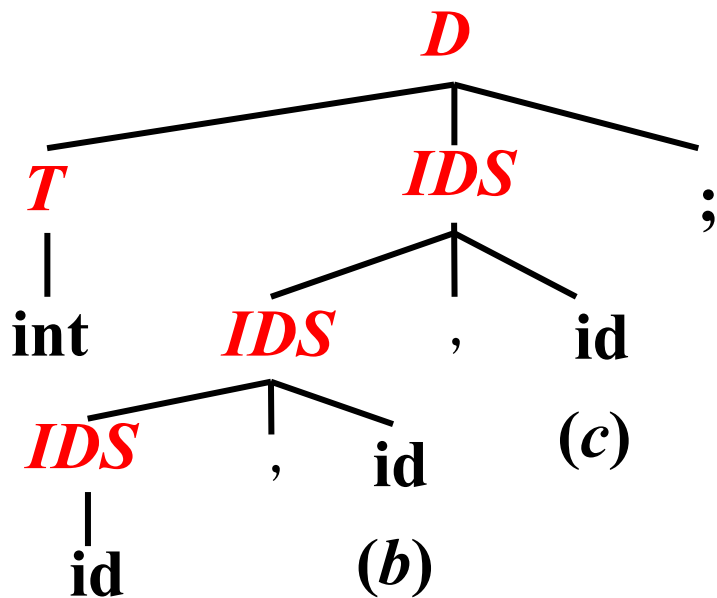
➤源代码：

`int a, b, c;`

➤词法分析器输出

`< int > < id, a > < , > < id, b >` (a)

`< , > < id, c > < ; >`



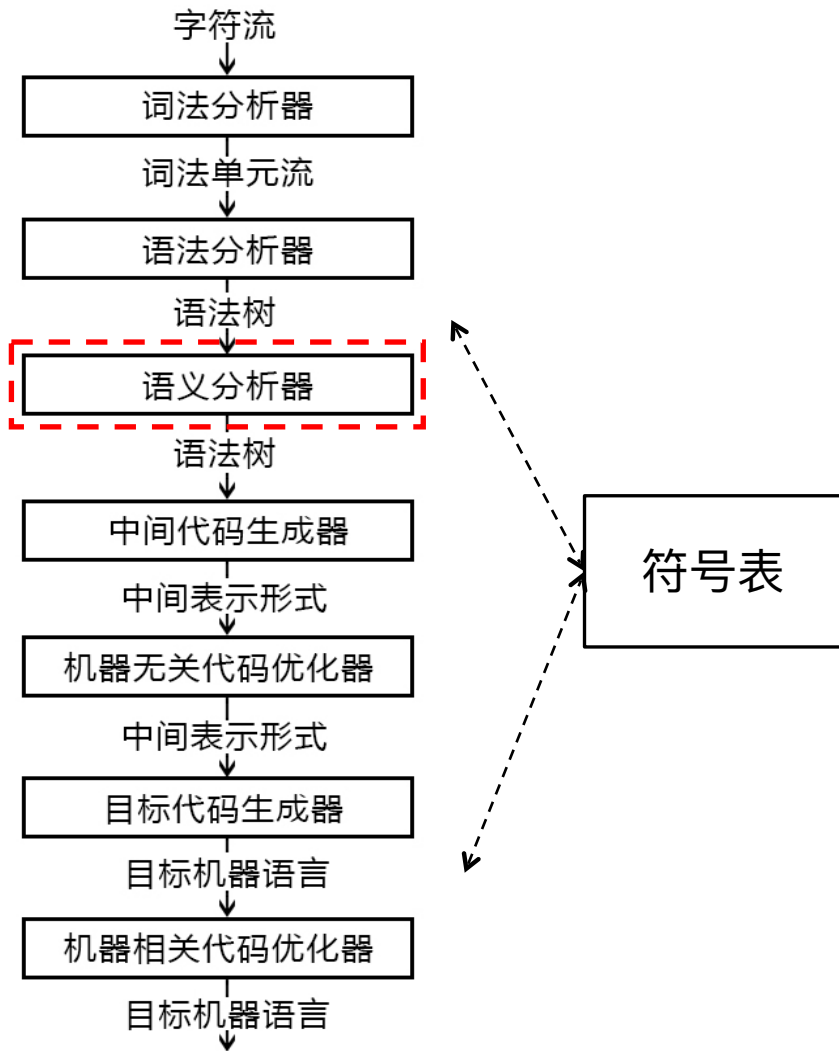
语法分析依赖语言的**文法**，上下文无关文法。

编译器的结构

任务：语义检查。

即对语法分析无法处理的语义规则进行检查。

- 信息收集
- 类型检查
- 语义检查



➤ 收集标识符的属性信息

➤ 种属 (Kind)

➤ 简单变量、复合变量（数组、记录、...）、过程、...

- 收集标识符的属性信息
 - 种属 (Kind)
 - 数据类型 (Type)
 - 整型、实型、字符型、布尔型、指针型、...

➤ 收集标识符的属性信息

➤ 种属 (Kind)

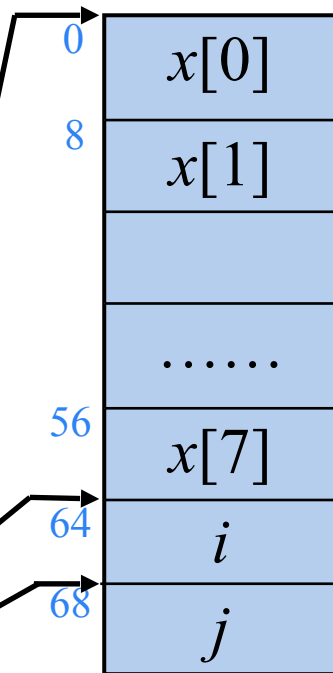
➤ 类型 (Type)

➤ 存储位置、长度

例：

```
begin
    real x[8];
    integer i, j;
    .....
end
```

名字	相对地址
<i>x</i>	0
<i>i</i>	64
<i>j</i>	68



➤ 收集标识符的属性信息

- 种属 (Kind)

- 类型 (Type)

- 存储位置、长度

- 值

- 作用域

- 参数和返回值信息

- 参数个数、参数类型、参数传递方式、返回值类型、...

语义分析的主要任务

➤ 收集标识符的属性信息

➤ 种属 (Kind)

➤ 类型 (Type)

➤ 存储位置、长度

➤ 值

➤ 作用域

➤ 参数和返回值信息

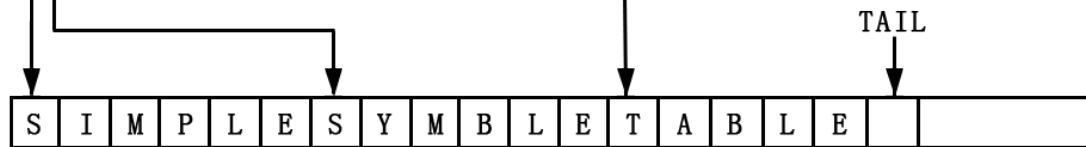
符号表中为什么要设计
字符串表这样一种数据结

【讨论1-2】符号表中NAME字段的设计

符号表(Symbol Table)

NAME	TYPE	KIND	VAL	ADDR
6	整	简变		
6	实	数组		
5	字符	常数		
:	:	:	:	:

字符串表



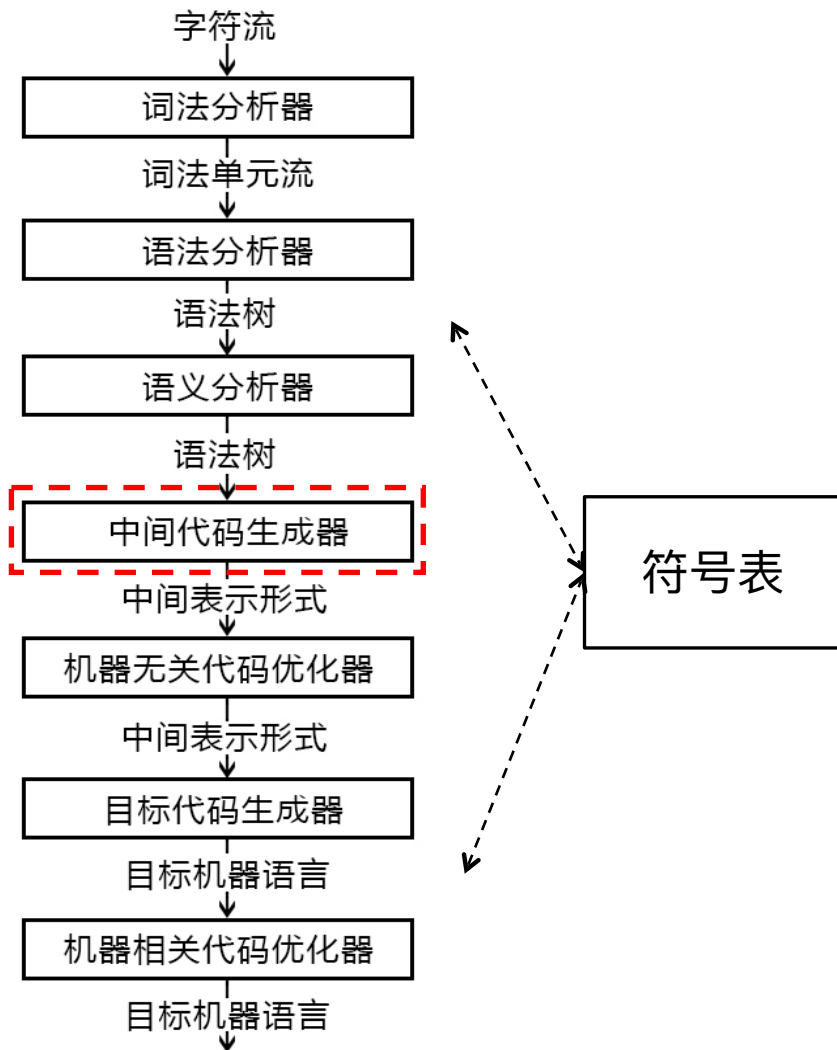
符号表是用于存放标识符的属性信息的数据结构

- 收集标识符的属性信息
- 类型检查
 - 运算分量类型不匹配
 - 操作符与操作数之间的类型不匹配
 - 数组下标不是整数
 - 对非数组变量使用数组访问操作符
 - 对非过程名使用过程调用操作符
- 语义检查
 - 变量或过程未经声明就使用
 - 变量或过程名重复声明

上下文无关文法无法实现的语义规范都要依赖语义检查。

➤ 中间代码的特点

- 简单规范
- 易于优化与转换
- 与机器无关
- 易于移植



➤各个阶段都可能使用多种中间表示

➤线性：三地址码（中间代码生成）

➤三地址码由类似于汇编语言的指令序列组成，每个指令最多有三个操作数

➤树型（图）：语法结构树/语法树（语法分析、语义分析）

➤混合型：数据流图（代码优化）

常用的三地址指令

序号	指令类型	指令形式
1	赋值指令	$x = y \text{ op } z$ $x = \text{op } y$
2	复制指令	$x = y$
3	条件跳转	if $x \text{ relop } y$ goto n
4	非条件跳转	goto n
5	参数传递	param x
6	过程调用 函数调用	call p, n $y = \text{call } p, n$
7	过程返回	return x
8	数组引用	$x = y[i]$
9	数组赋值	$x[i] = y$
10	地址及 指针操作	$x = \& y$ $x = * y$

三地址指令：一个运算符，三个地址；
三地址：(最多)两个运算分量地址，
一个结果分量地址。

地址可以具有如下形式之一

- 源程序中的名字 (*name*)
- 常量 (*constant*)
- 编译器生成的临时变量 (*temporary*)

三地址指令的四元式表示

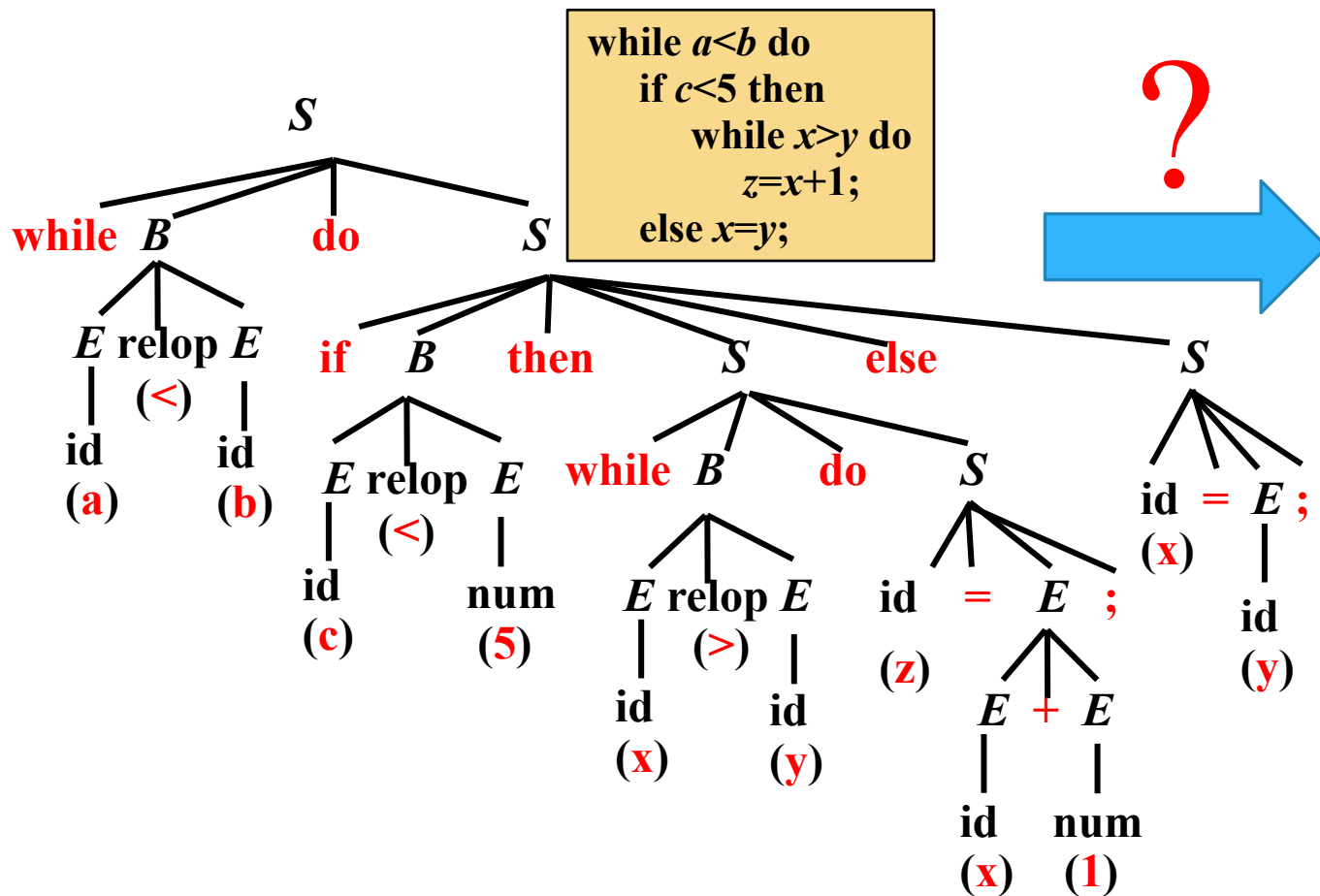
➤ $x = y \text{ op } z$	(op , y , z , x)
➤ $x = \text{op } y$	(op , y , _ , x)
➤ $x = y$	(= , y , _ , x)
➤ if $x \text{ relop } y$ goto n	(relop , x , y , n)
➤ goto n	(goto , _ , _ , n)
➤ param x	(param , _ , _ , x)
$y = \text{call } p, n$	(call , p , n , y)
return x	(return , _ , _ , x)
➤ $x = y[i]$	(=[, y , i , x)
$x[i] = y$	([]= , y , x , i)
➤ $x = \&y$	(& , y , _ , x)
$x = *y$	(=* , y , _ , x)
$*x = y$	(*= , y , _ , x)



三地址指令序列唯一确定了
运算完成的顺序

- 第一个分量是**操作符**
- 第二、三个分量是**操作数**
- 第四个分量是**结果地址**

中间代码生成的例子



100: ($j <$, a , b , 102)
 101: (j , - , - , 112)
 102: ($j <$, c , 5 , 104)
 103: (j , - , - , 110)
 104: ($j >$, x , y , 106)
 105: (j , - , - , 100)
 106: ($+$, x , 1 , t_1)
 107: ($=$, t_1 , - , z)
 108: (j , - , - , 104)
 109: (j , - , - , 100)
 110: ($=$, y , - , x)
 111: (j , - , - , 100)
 112:

➤ 代码优化

➤ 为改进代码所进行的等价程序变换，使其运行得更快一些、占用空间更少一些，或者二者兼顾

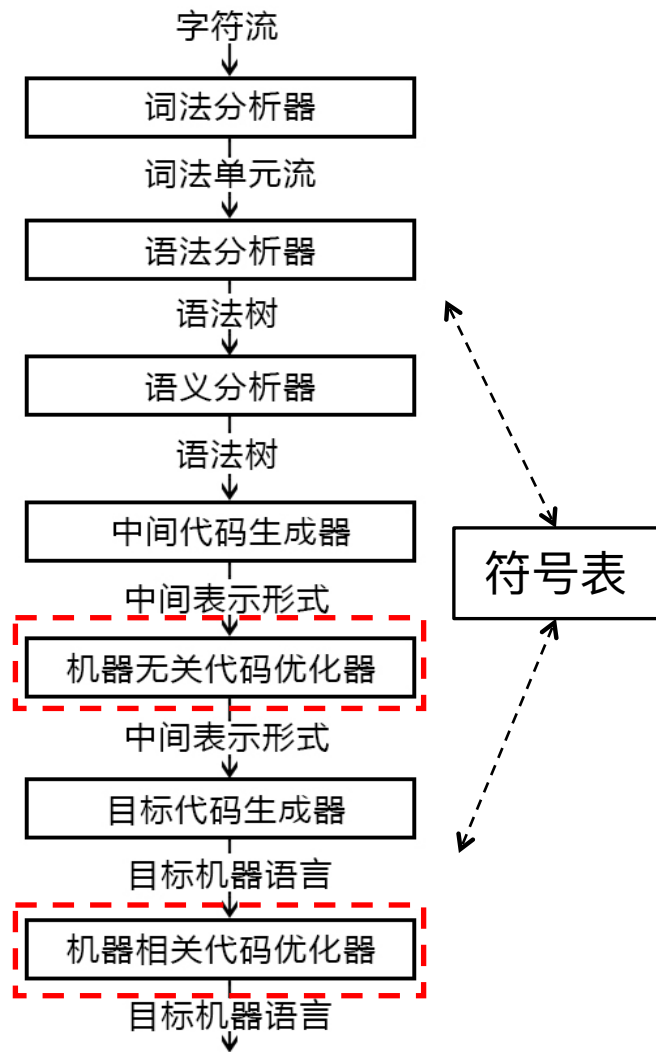
➤ 最基本的优化原则：

语义等价

➤ 优化分类

➤ 机器无关优化

➤ 机器相关优化



与机器无关的优化

➤ 局部优化

- 常量合并：常数运算在编译期间完成，如 $8+9*4$
- 公共子表达式的提取：避免重复计算相同表达式

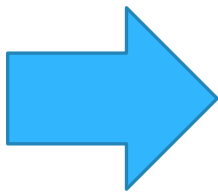
➤ 循环优化

- 强度削减
 - 用较快的操作代替较慢的操作
- 代码外提
 - 将循环不变计算移出循环

与机器无关的优化

例：将循环不变计算移出循环

```
z=read();  
for(i=1; i<n; i++)  
{  
    y=read();  
    t0=23;  
    t1=z*t0;  
    t2= y+t1;  
    x=t2;  
}
```



```
z=read();  
t0=23;  
t1=z*t0;  
for(i=1; i<n; i++) {  
    y=read();  
    t2= y+t1;  
    x=t2;  
}
```

➤寄存器的利用

- 将常用量放入寄存器，以减少访问内存的次数

➤体系结构

- MIMD、SIMD、SPMD、向量机、流水机

➤存储策略

- 根据算法访存的要求安排：Cache、并行存储体系——减少访问冲突

➤任务划分

- 按运行的算法及体系结构，划分子任务(MPMD)

编译器的结构

➤ 目标代码生成

➤ 输入：源程序的**中间代码表示**

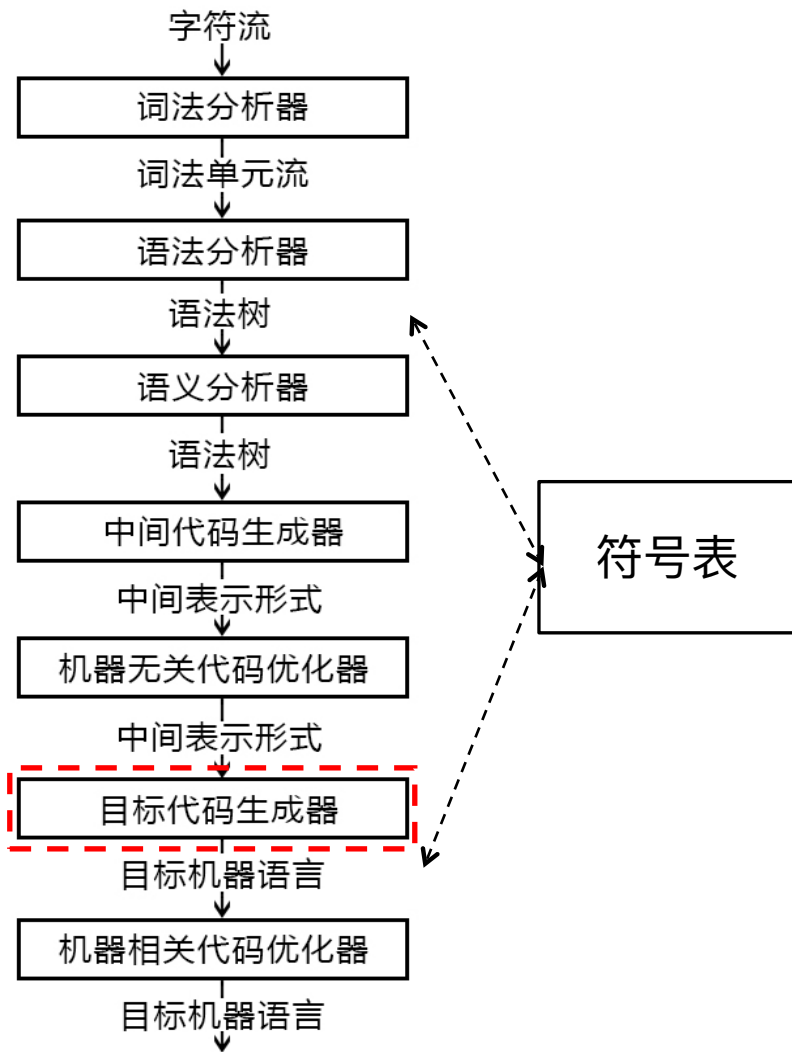
➤ 输出：**目标语言程序**

➤ 重要任务：

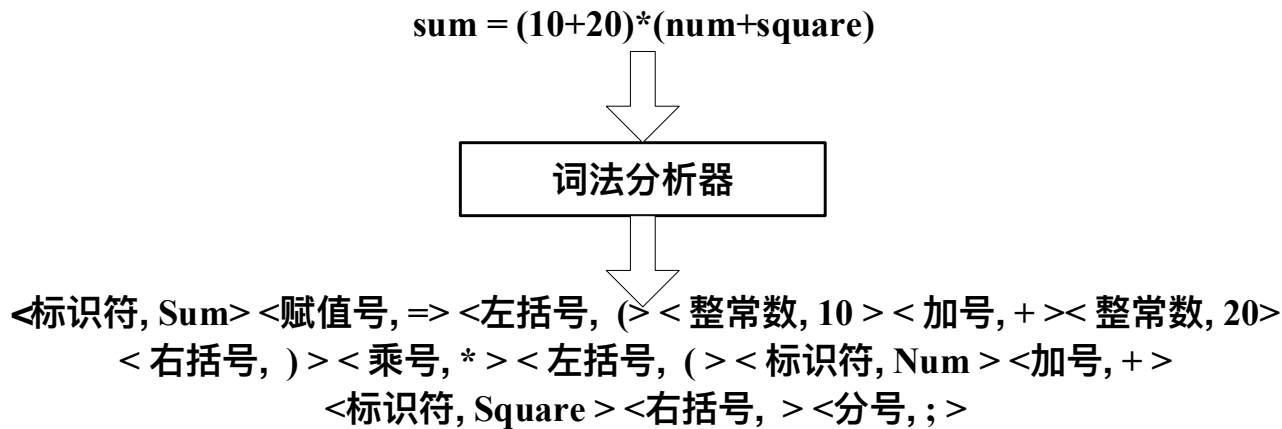
➤ **指令选择**

➤ **寄存器分配**

➤ **指令排序**

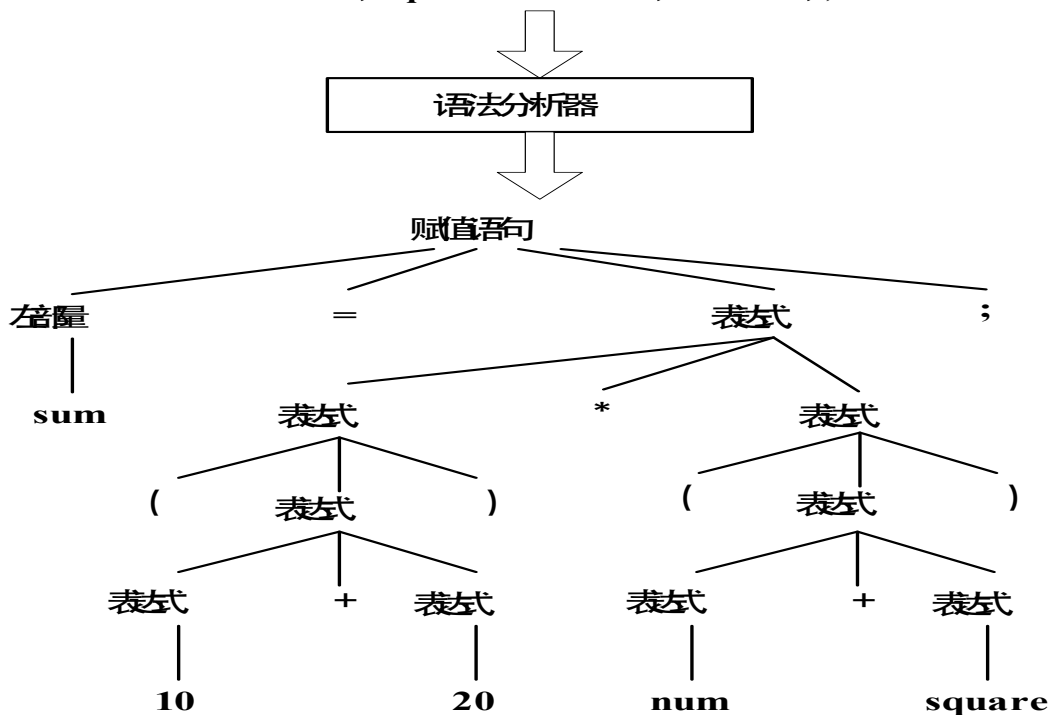


语句`sum=(10+20)*(num+square);`的翻译过程

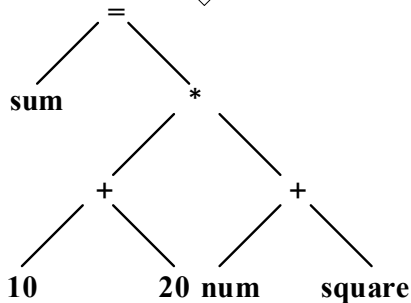
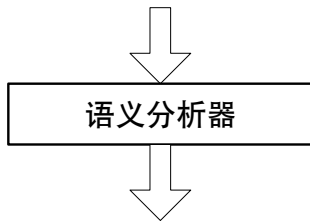
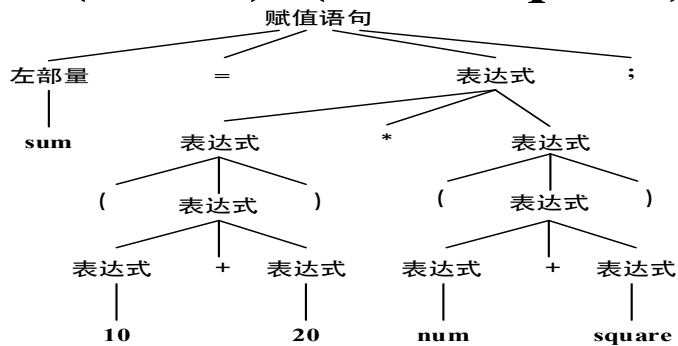


语句 $\text{sum}=(10+20)*(\text{num}+\text{square});$ 的翻译过程

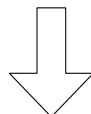
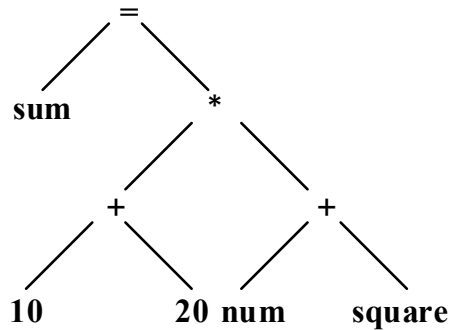
<标识符, Sum> <赋值号, => <左括号, (> <整常数, 10> <加号, +> <整常数, 20>
<右括号,)> <乘号, *> <左括号, (> <标识符, Num> <加号, +>
<标识符, Square> <右括号,)> <分号, ;>



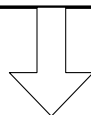
语句 $\text{sum}=(10+20)*(\text{num}+\text{square});$ 的翻译过程



语句 $\text{sum}=(10+20)*(\text{num}+\text{square});$ 的翻译过程



中间代码生成器



$T_1=10+20$

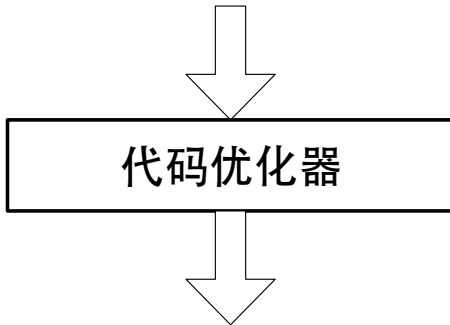
$T_2=\text{num}+\text{square}$

$T_3=T_1*T_2$

$\text{sum}=T_3$

语句 $\text{sum}=(10+20)*(\text{num}+\text{square});$ 的翻译过程

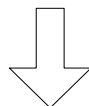
$T_1=10+20$ $T_2=\text{num}+\text{square}$ $T_3=T_1*T_2$ $\text{sum}=T_3$



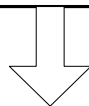
$T_1 = \text{num} + \text{square}$ $\text{sum} = 30 * T_1$

语句 $\text{sum}=(10+20)*(\text{num}+\text{square});$ 的翻译过程

$T_1 = \text{num} + \text{square}$ $\text{sum} = 30 * T_1$



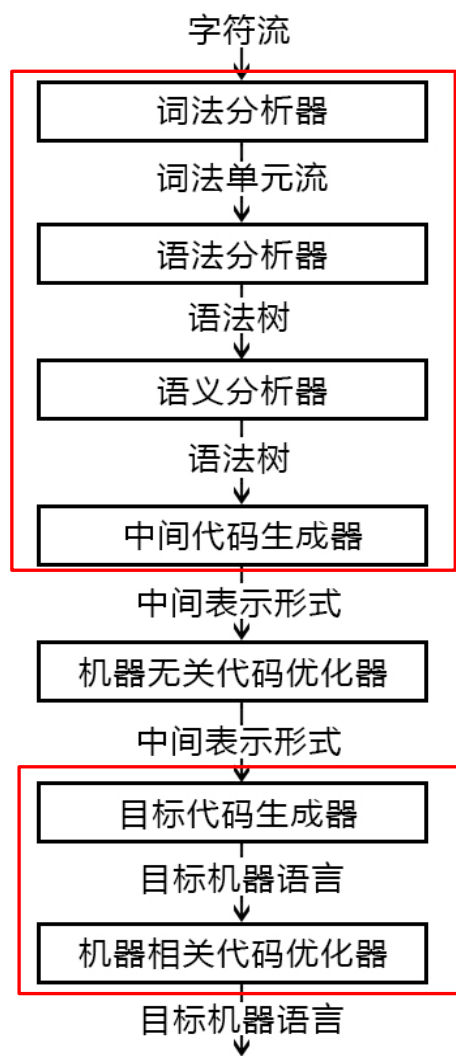
目标代码生成器

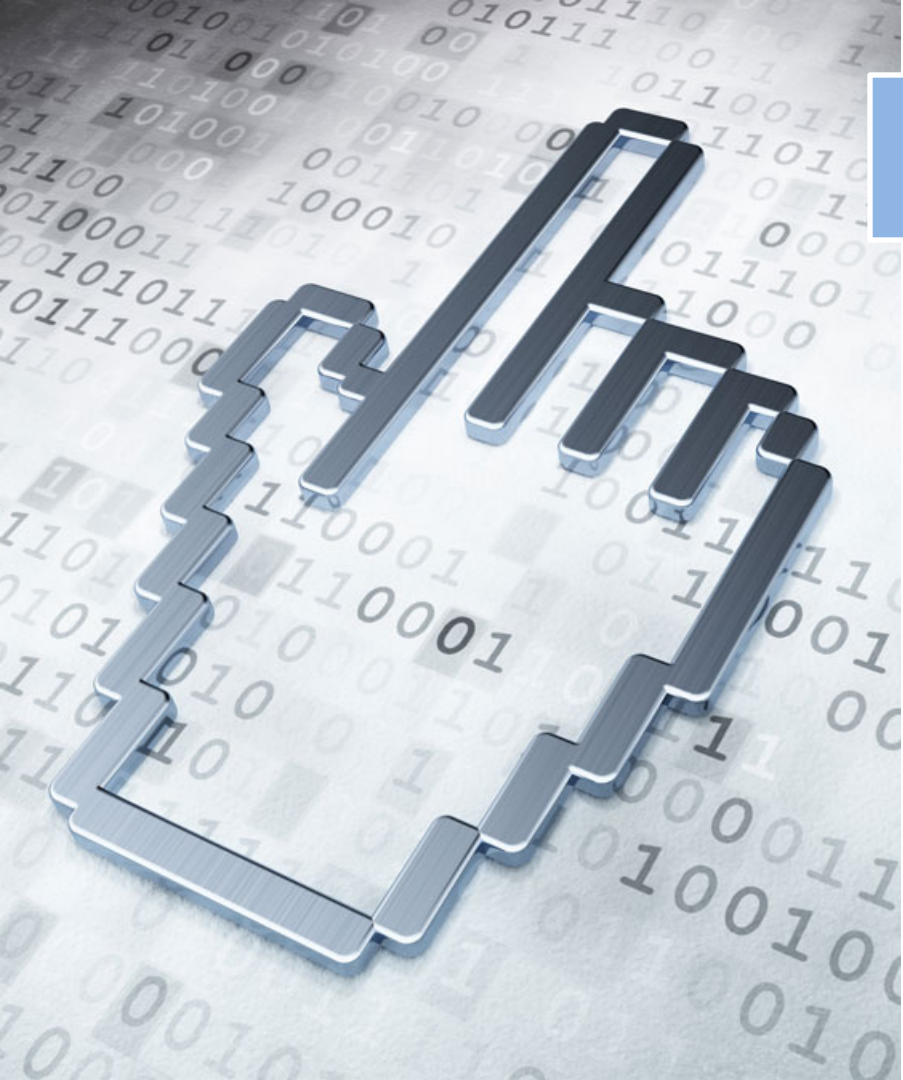


MOV num, R1 MOV square, R2 ADD R2, R1 MUL 30, R1 MOV R1, sum

编译程序的组织

- **遍**：编译程序对源程序或中间结果的一次完整扫描处理称为一遍。
- 通常**前端**与**后端**是独立的遍，便于移植。
- **前端所有任务可一遍扫描完成**，通常以**语法分析器**为中心。但复杂，模块化不好。
- 现代编译器通常将词法和语法组织为独立的遍，语义分析和中间代码生成为独立的遍。
 - 模块化好，需足足够空间存储语法分析树。
- 优化阶段通常需要**若干遍扫描**才能实现优良的优化





本章内容

1.1 什么是编译

1.2 编译系统的结构

1.3 编译器的生成

1.4 为什么要学习编译原理

1.5 编译技术的应用

1.3 编译程序的生成 *

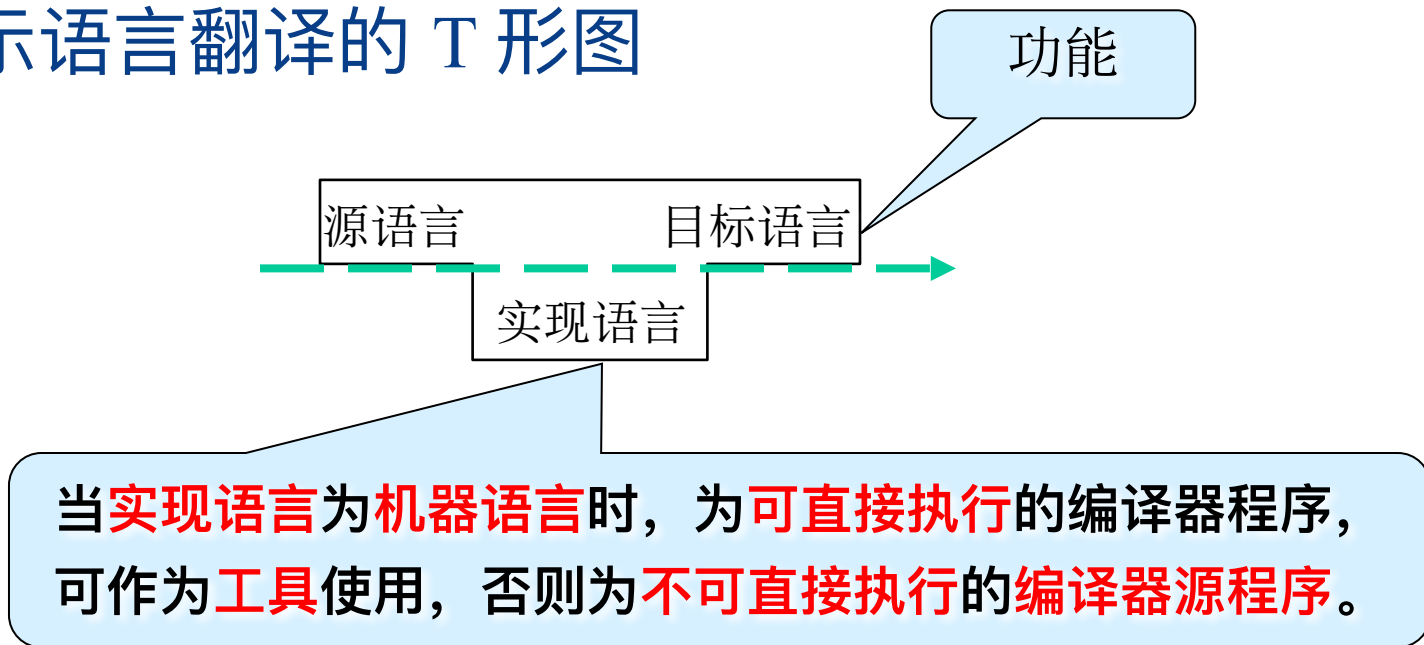
- 第一个高级语言编译器是怎样被编译的?
 - 直接用可运行的代码从头到尾编制——太费力!
- 如何利用A机器上的编译器开发B机器上的编译器?
- 如何继续开发新语言的编译器?

➤ 构建策略

- 自展
- 移植
- 自动生成程序

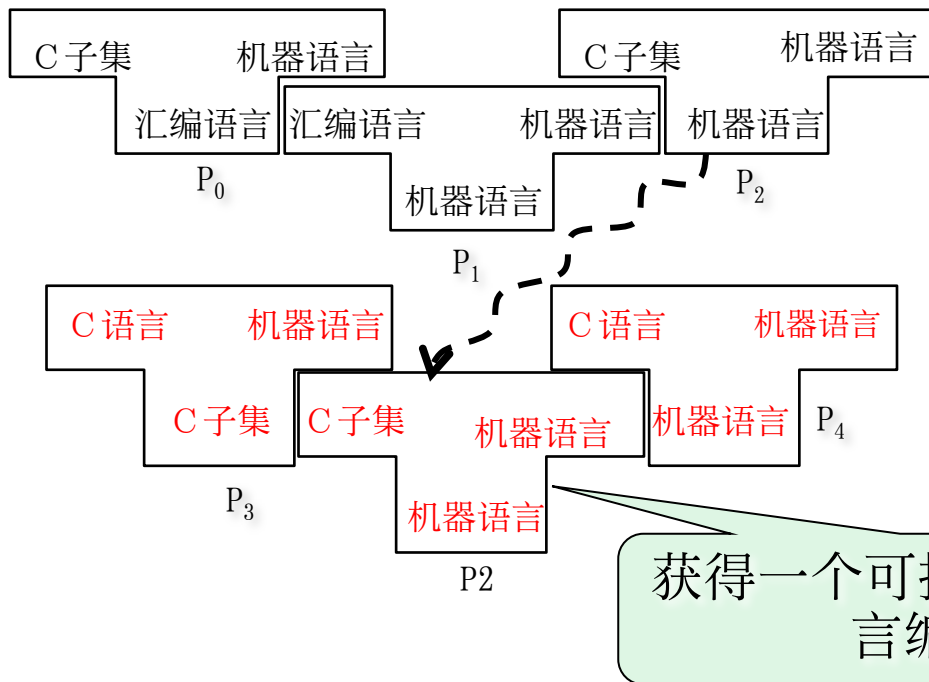
1. T形图

➤ 表示语言翻译的 T 形图



2. 自展

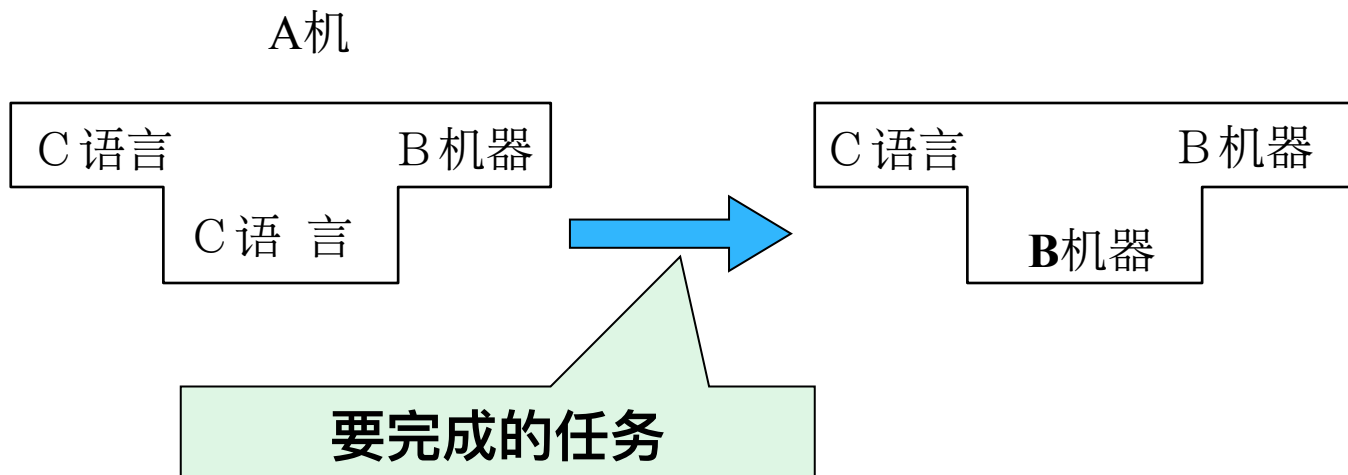
➤问题一：如何利用汇编器实现第一个C语言编译器？

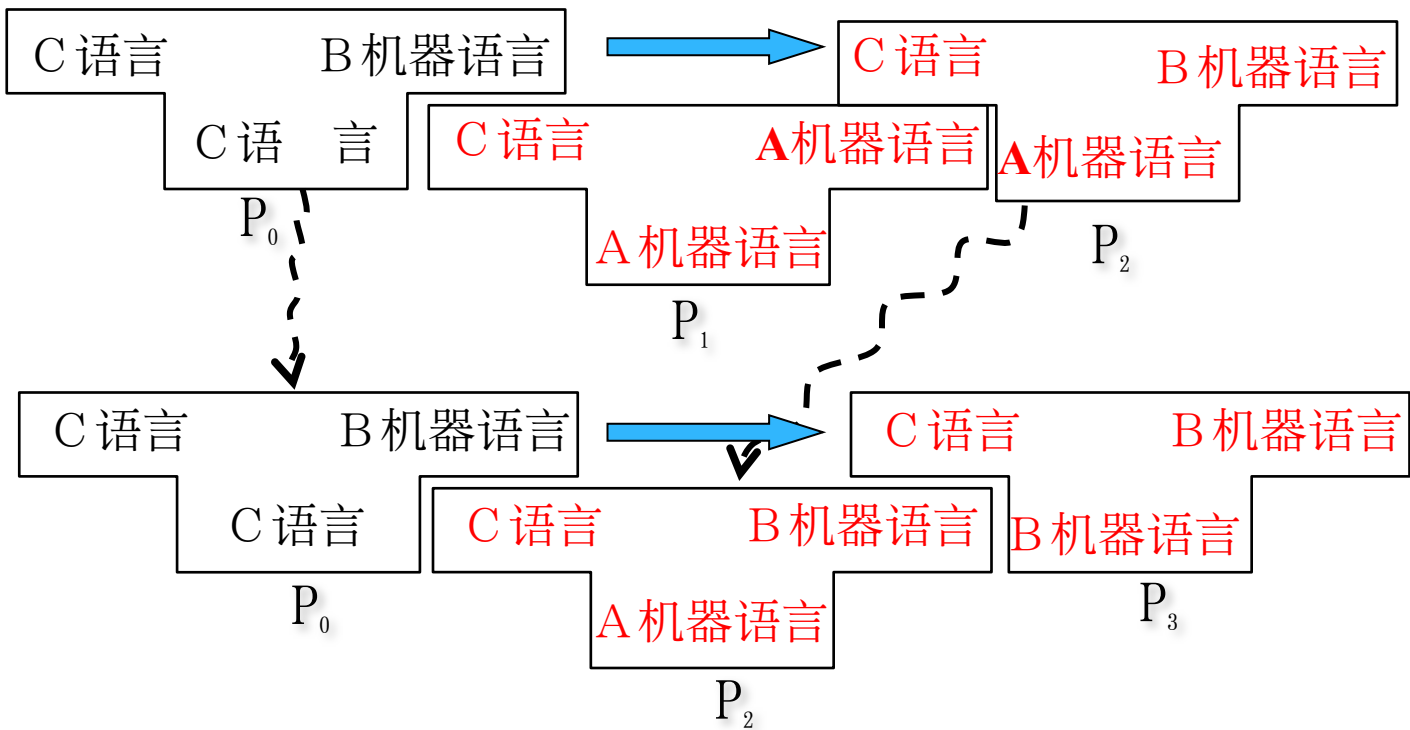


1. 用汇编语言实现一个 C子集的编译程序(P_0 —人)
2. 用汇编程序(P_1)处理该程序,得到(P_2 :可直接运行)
3. 用C子集编制 C语言的编译程序(P_3 —人)
4. 用 P_2 编译 P_3 , 得到 P_4

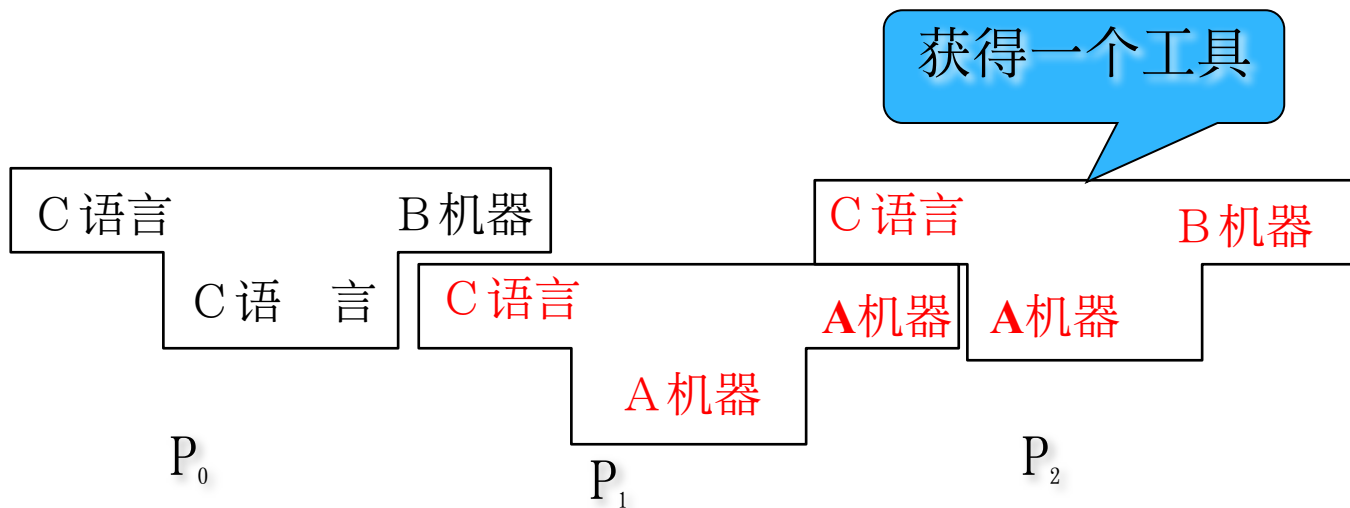
3.移植

- 问题二：A机上有一个C语言编译器，是否可利用此编译器实现B机上的C语言编译器？

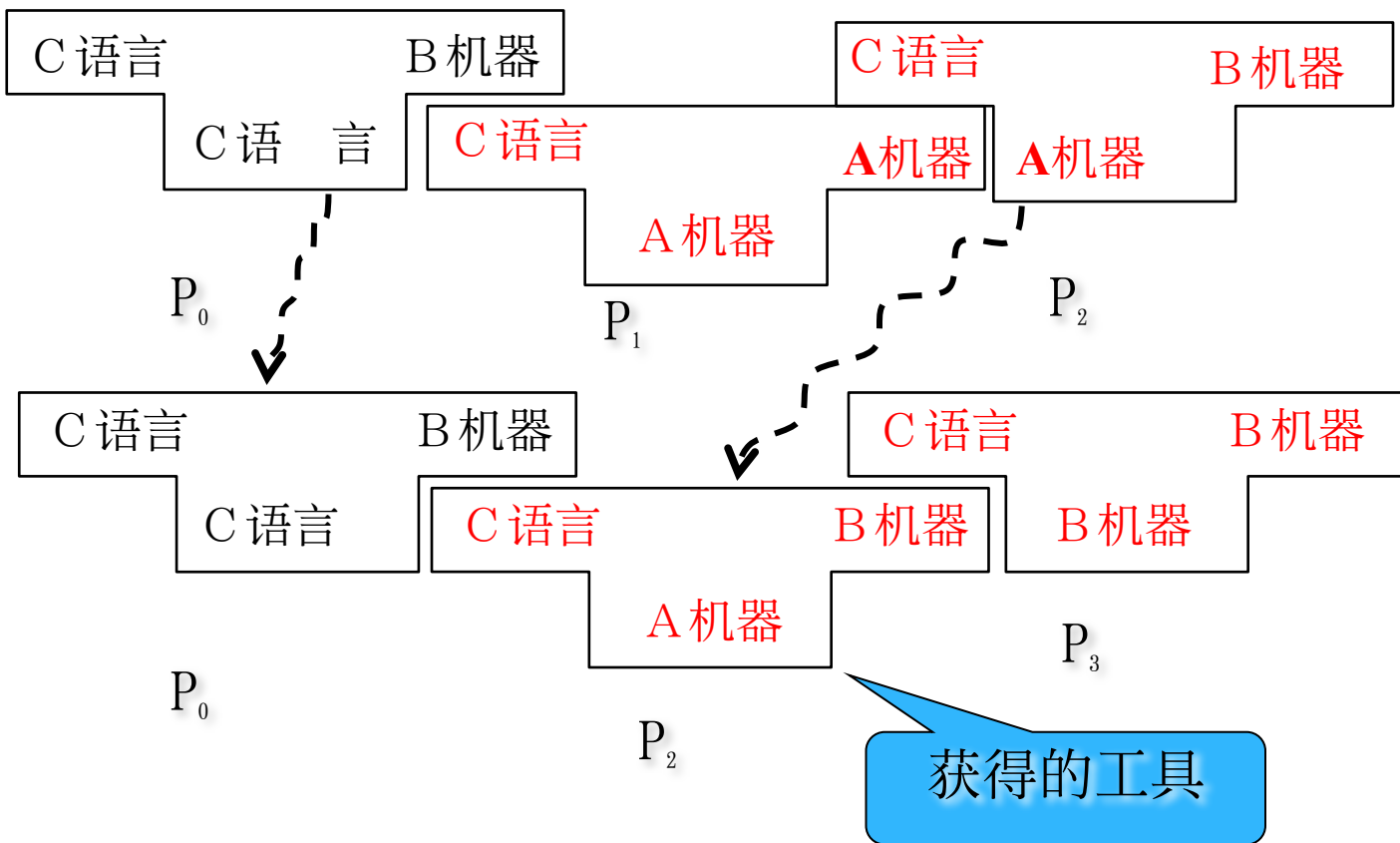




2)问题的解决办法



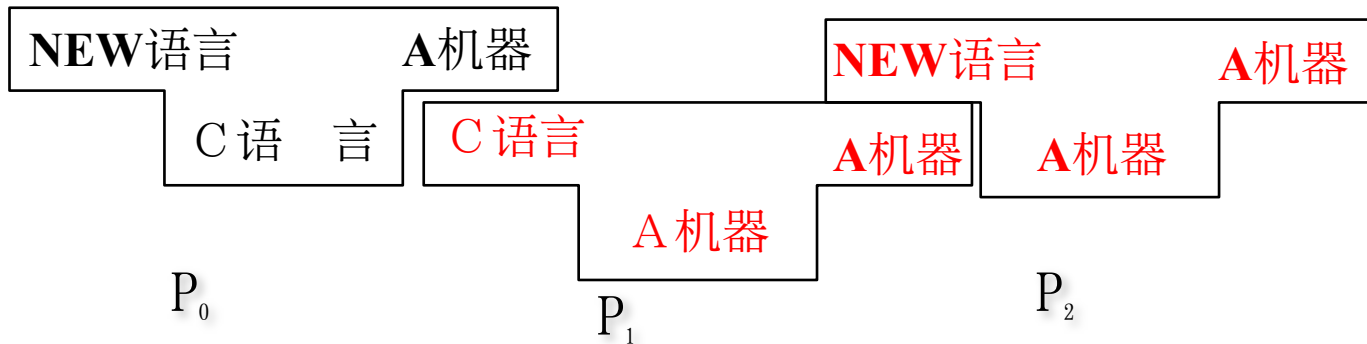
1. (人)用C语言编制B机的C编译程序 $P_0(C \rightarrow B)$
2. (A机的C编译器 P_1)编译 P_0 , 得到在A机上可运行的 $P_2(C \rightarrow B)$

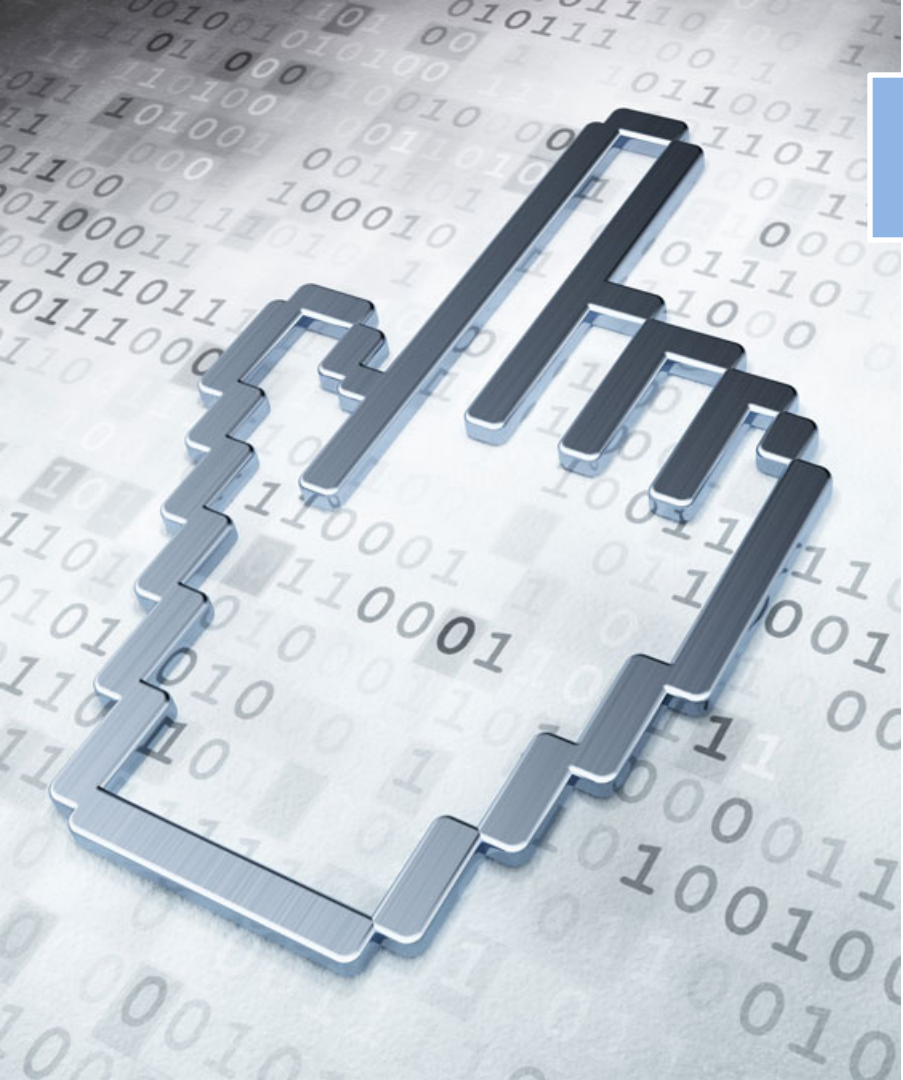


3.(A 机的P₂)编译P₀, 得到在B机上可运行的P₃(C→B)

4.本机编译器的利用

- 问题三： A机上有一个C语言编译器，现要实现一个新语言NEW的编译器？
- 用C编写NEW的编译，并用C编译器编译它





本章内容

1.1 什么是编译

1.2 编译系统的结构

1.3 编译器的生成

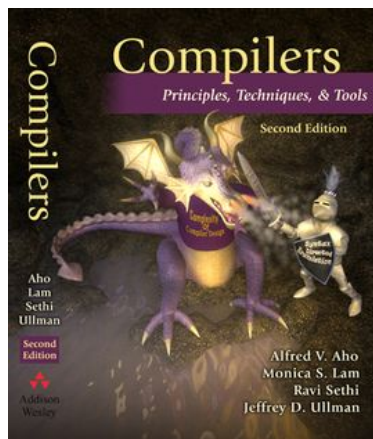
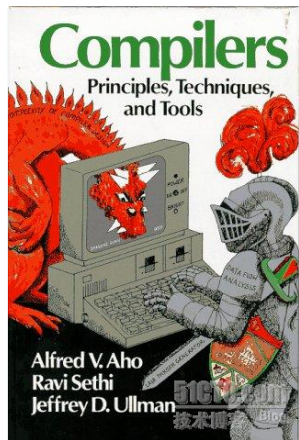
1.4 为什么要学习编译原理

1.5 编译技术的应用

1.4 为什么要学习编译原理

编写编译器的原理和技术具有十分普遍的意义，以至于在每个计算机科学家的研究生涯中，本课程中的原理和技术都会反复用到。

——Alfred V.Aho



ACM图灵奖

<https://amturing.acm.org/bysubject.cfm>

编程语言、编译相关的获奖者
是最多的 约占1/3

Analysis of Algorithms Artificial Intelligence
Combinatorial Algorithms Compilers Computational Complexity
Computer Architecture Computer Hardware Cryptography
Data Structures Databases Education Error Correcting Codes Finite Automata Graphics
Interactive Computing Internet Communications List Processing Numerical Analysis
Numerical Methods Object Oriented Programming Operating Systems Personal Computing
Program Verification Programming
Programming Languages Proof Construction Software
Theory Software Engineering
Verification of Hardware and Software Models Computer Systems Machine Learning
Parallel Computation



理论、技术与实践应用结合的典范

实践层

编译器

程序设计

技术层

词法、语法
与语义分析

中间代码生
成

代码优化

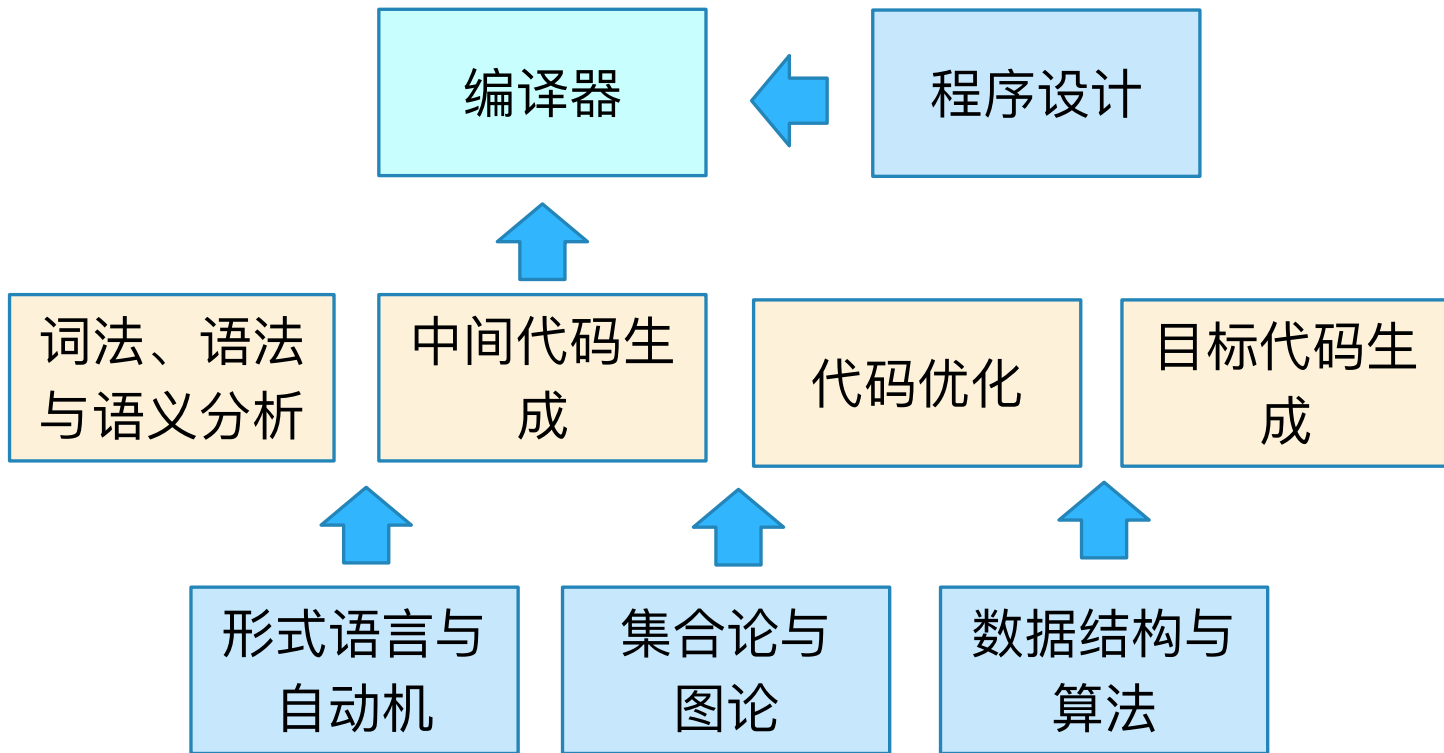
目标代码生
成

理论层

形式语言与
自动机

集合论与
图论

数据结构与
算法



- 所涉及的理论和方法在很多领域都会被用到
 - 自然语言处理、模式识别、人工智能、.....
 - 句法分析、机器翻译、输入法、text2SQL解析技术
- 是计算机科学中理论、技术与实践应用结合的典范，计算机系统能力培养的典型载体。
- 更深刻地理解高级语言程序的内部实现机制
 - 有利于提升正确、高效的编程能力，提升程序代码的执行效率，降低代码隐含错误、缺陷和漏洞。
- 深化和实践计算机科学解决复杂问题的一般方法学；
 - “形式化→自动化”；

➤ 人工智能技术的再次兴起

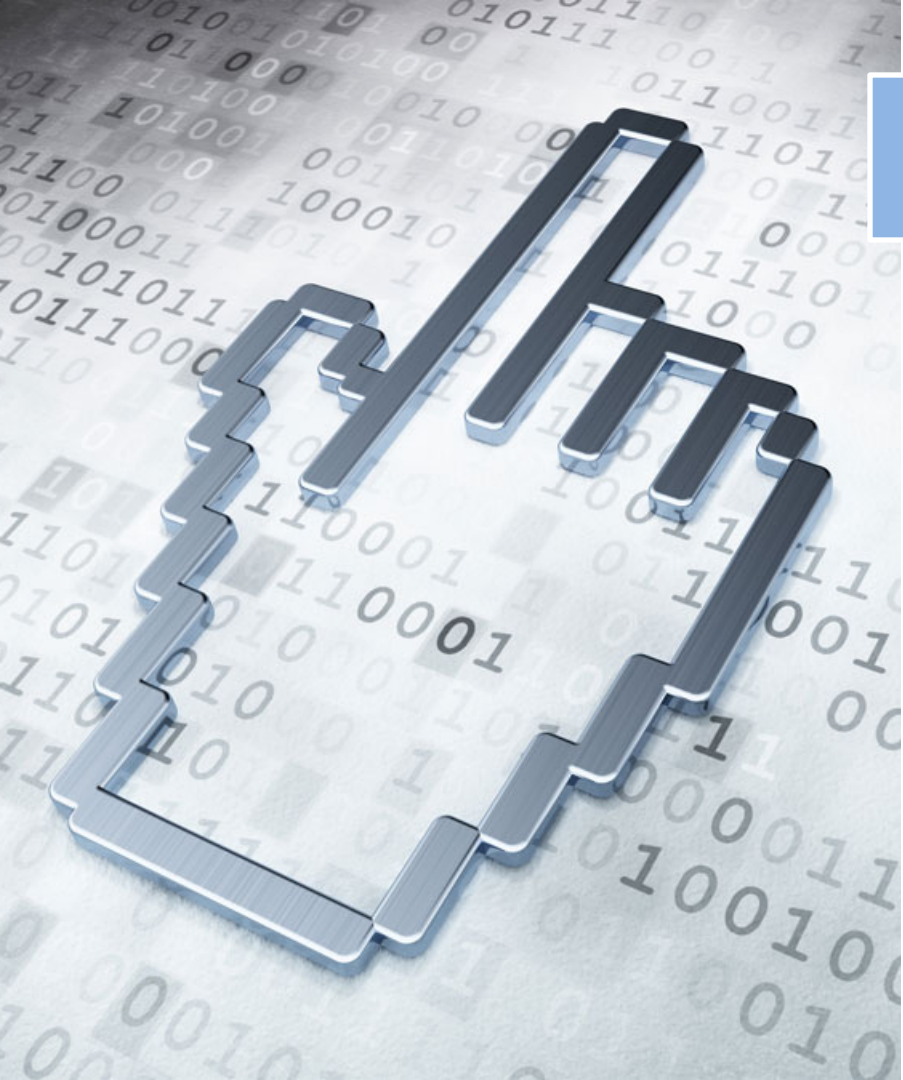
- 人工智能加速芯片
 - 人工智能算法开发
- 对程序设计语言和编译
提出新的更高要求

➤ 面对美国芯片封锁，芯片自主可控迫在眉睫

- 芯片自主研发与生产方兴未艾
 - 实现芯片自给，从芯片大国走向芯片强国
- 研发和生态发展
都需要编译技术

➤ 国产编程语言和编译器等核心基础系统软件自主需求

➤ 面向应用/硬件的领域特定语言DSL软硬件协同的编译系统优化



本章内容

1.1 什么是编译

1.2 编译系统的结构

1.3 编译器的生成

1.4 为什么要学习编译原理

1.5 编译技术的应用

1.5 编译技术的应用

- 很多应用软件都会用到编译技术
 - 开发新语言（**DSL (Domain Specific Language)** 领域特定语言），扩展语言新特性
 - 语言升级新体系结构的优化支持（并行和内存层次结构）
- 提高软件开发效率的工具
 - 程序错误检测和定位：数据流分析技术
 - 内存管理错误检测
 - 专用优化编译器
- 新计算机体系结构的设计
 - 计算机系统的整体性能也依赖编译器对其特性支持的程度

1.4 编译技术的应用

- 结构化编辑器 (Structure editors)
 - 引导用户在语言的语法约束下编制程序
 - 能自动地提供关键字和与其匹配的关键字

1.4 编译技术的应用

- 结构化编辑器 (Structure editors)
- 智能打印机 (Pretty printers)
 - 对程序进行分析，打印出结构清晰的程序
 - 注释以一种特殊的字体打印
 - 根据各个语句在程序的层次结构中的嵌套深度进行缩进

1.4 编译技术的应用

- 结构化编辑器 (Structure editors)
- 智能打印机 (Pretty printers)
- 静态检测器 (Static checkers)
 - 静态定位程序中的错误
 - 释放空指针或已释放过的指针
 - 检测出程序中永远不能被执行的语句

1.4 编译技术的应用

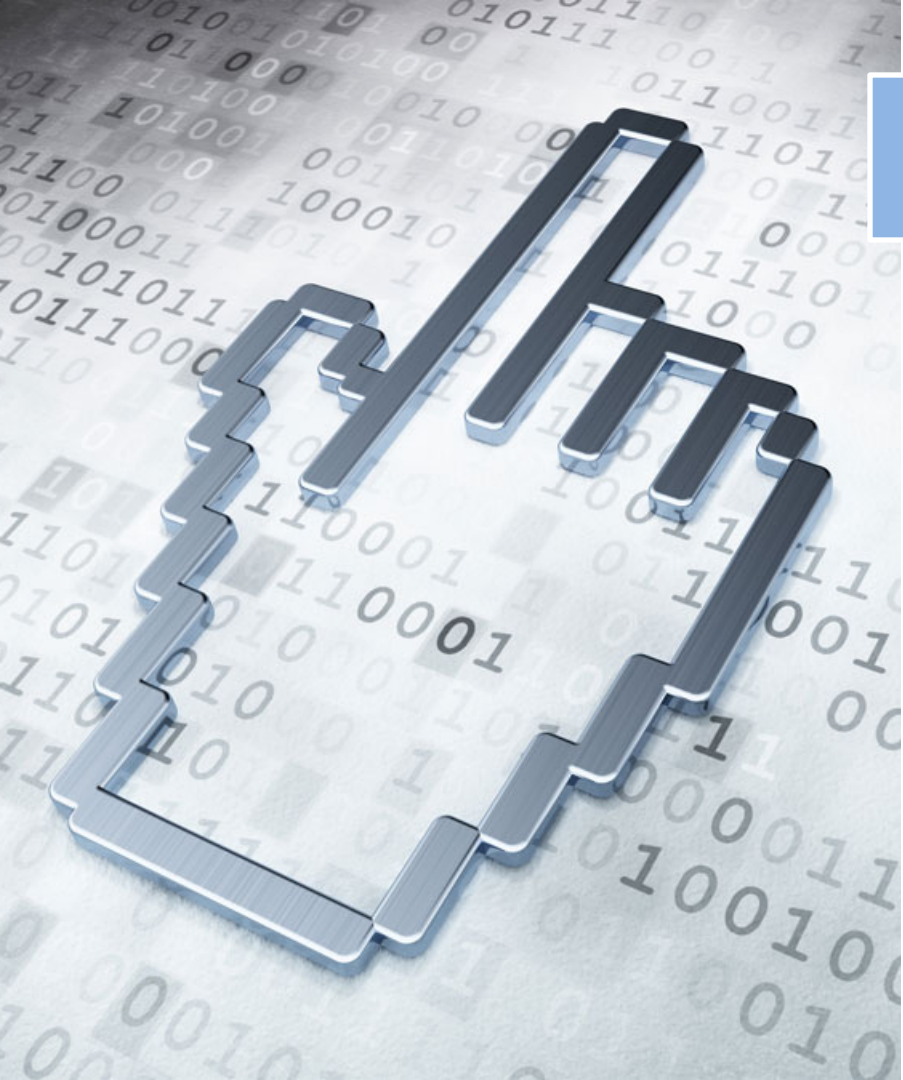
- 结构化编辑器 (Structure editors)
- 智能打印机 (Pretty printers)
- 静态检测器 (Static checkers)
- 文本格式器 (Text formatters)
 - 文本格式器处理的字符流中除了需要排版输出的字符以外，还包含一些用来说明字符流中的段落、图表或者上标和下标等数学结构的命令

1.4 编译技术的应用

- 结构化编辑器 (Structure editors)
- 智能打印机 (Pretty printers)
- 静态检测器 (Static checkers)
- 文本格式器 (Text formatters)
- 数据库查询解释器 (Database Query Interpreters)
 - 数据库查询语句由包含了关系和布尔运算的谓词组成。查询解释器把这些谓词翻译成数据库命令，在数据库中查询满足条件的记录。

1.4 编译技术的应用

- 结构化编辑器 (Structure editors)
- 智能打印机 (Pretty printers)
- 静态检测器 (Static checkers)
- 文本格式器 (Text formatters)
- 数据库查询解释器 (Database Query Interpreters)
- 高级语言的翻译工具



本章小结

什么是编译

编译系统的结构

编译器的生成方法

为什么要学习编译原理

编译技术的应用

- | | |
|-------------|-------|
| 1. 绪论 | (2学时) |
| 2. 语言及其文法 | (2学时) |
| 3. 词法分析 | (3学时) |
| 4. 语法分析 | (9学时) |
| 5. 语法制导翻译 | (6学时) |
| 6. 中间代码生成 | (7学时) |
| 7. 运行时的存贮组织 | (3学时) |
| 8. 代码优化 | (6学时) |
| 9. 代码生成 | (2学时) |



结束

